

Architettura degli elaboratori

[Appunti di Piai Luca]

Academic Year 2021-2022

Indice

1	Introduzione al processore ARM	3
1.1	Il linguaggio assembly	3
1.1.1	Caratteristiche delle istruzioni	3
1.2	Metodi di indirizzamento	4
1.3	Istruzioni in ARM	5
1.3.1	Istruzioni condizionate	5
1.3.2	Sintassi delle istruzioni	5
1.3.3	Miscellanea	6
2	Assembly in ARM	7
2.1	Struttura di un programma assembly	7
2.1.1	Costrutti di base	8
2.2	Strutture dati	8
2.2.1	Array	8
2.2.2	Matrici	9
2.2.3	Stack	10
2.2.4	Liste concatenate	11
3	Subroutine e funzioni	13
3.1	Definire una funzione	13
3.2	Passaggio di parametri	14
3.3	Allocazione dinamica	16
3.3.1	Subroutine rientranti	16
3.3.2	Allocazione dinamica ARM e HLL	17
3.4	Esempi di chiamate a subroutine in ARM	21
3.5	Costruzione di un file eseguibile	23
3.5.1	Moduli sorgenti	23
3.5.2	L'assembler	24
3.5.3	Il linker	26
3.5.4	Il loader	27
3.5.5	Le librerie	27
4	Memorie cache, paging, memoria virtuale	28
4.1	Gerarchie di memoria e principio di località di riferimento	28
4.2	Struttura cache e memoria	29
4.3	Tipi di cache	29
4.3.1	Mappatura diretta	29
4.3.2	Completamente associativa	30
4.3.3	Set-associativa a k vie	31
4.4	Algoritmi di rimpiazzo	32
4.5	Gerarchia multilivello	32
4.6	Gestione della memoria	32
4.6.1	Paginazione	33
4.6.2	Segmentazione	34
4.7	Memoria virtuale	34
4.7.1	Paginazione e memoria virtuale	35
5	Input Output	37
5.1	I/O programmato	38
5.2	I/O a interrupt	38
5.3	I/O con DMA	40
6	Organizzazione del processore	42

1 Introduzione al processore ARM

L'architettura di un elaboratore è la parte visibile ad un programmatore. In particolare l'architettura include tutto quello che serve ad un programmatore per controllare una macchina, per esempio accesso ai dati in memoria e ai registri, istruzioni e molto altro.

Nel nostro caso usiamo l'**architettura ARM**, esistono altre architetture come per esempio x86 oppure MIPS, che condividono elementi comuni.

1.1 Il linguaggio assembly

Caratteristiche del linguaggio:

- **Istruzioni macchina:** ogni sequenza di 4 byte rappresenta un'*istruzione macchina*.
- Corrispondenza 1 a 1 tra istruzioni e codice ARM, ogni istruzione viene codificata in una stringa alfanumerica nel *linguaggio assembly*.

Registri accessibili:

- **PC:** program counter;
- **SP:** stack pointer;
- **FP:** frame pointer;
- Registri generici: R1,R2,..., il programmatore può scegliere come utilizzarli;
- Registro di stato: contiene le informazioni sullo stato del processore.

In particolare il *registro di stato* è un registro speciale contenente informazioni sullo stato della macchina organizzati in *campi*, dove i campi sono insiemi bit che rappresentano diverse situazioni.

Le flag sono campi di un bip. Le principali flag sono:

- **N:** negative, se $N = 1$ significa che l'ultimo risultato era negativo, altrimenti $N = 0$;
- **Z:** zero, se $Z = 1$ l'ultimo risultato ottenuto era zero, altrimenti $Z = 0$;
- **C:** carry, bit di riporto (ovvero il 32-esimo);
- **V:** overflow, indica se l'ultima operazione aritmetica ha causato un overflow.

Importante specificare che **ogni istruzione può modificare alcuni bit di flag**.

1.1.1 Caratteristiche delle istruzioni

Tipi di istruzioni:

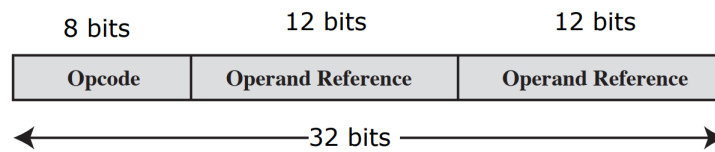
- Aritmetico/logiche;
- Trasferimento dati;
- Input/Output;
- Trasferimento di controllo.

Per quanto riguarda il *trasferimento dati*, ARM, non ha istruzioni per spostamenti da memoria a memoria.

Le istruzioni di *trasferimento di controllo* determinano le prossime istruzioni da eseguire. Sostanzialmente il program counter, dopo l'esecuzione di un'istruzione, viene incrementato $PC+4$, ma in molti casi è necessario cambiare l'indirizzo della prossima istruzione (per i cicli, if, subroutine,...): $PC = \text{nuovo valore}$. In questi casi utilizzeremo spesso i salti incondizionati (*branch*), salti istruzione (*skip*) e le chiamate a procedure.

1.2 Metodi di indirizzamento

Figura 1: Rappresentazione operandi



Ogni istruzione è rappresentata da una sequenza di bit organizzati in campi. Per indicare dove si trova un operando sfruttiamo i **metodi di indirizzamento**:

- **immediato**: il valore, rappresentato in complemento a 2, viene indicato direttamente nel campo operando. Questo metodo non richiede accessi (se non il fetch come tutte le istruzioni) in memoria, ma possiamo memorizzare valori fino a 12 bit. Viene usato per definire costanti o variabili. Per esempio:

```
1  MOV R0, #5
2  ADD R1, R0, #0xD
```

il simbolo # viene utilizzato per indicare il *valore immediato*;

- **diretto di registro**: viene specificato un registro che contiene l'operando. Non richiede accessi in memoria oltre al fetch, sono richiesti pochi bit per il campo indirizzi, ma lo spazio degli indirizzi è limitato a causa del numero di registri. Per esempio:

```
1  MOV R0, #5
2  ADD R1, R0, #0xD
```

i registri R0, R1 contengono i valori;

- **indiretto di registro**: viene specificato un registro che contiene l'indirizzo in memoria dell'operando. Richiede un accesso in memoria oltre al fetch. Per esempio:

```
1  LDR R0, [R2]
```

tra le parentesi quadre inseriamo sempre indirizzi di memoria;

- **con offset**: l'istruzione è composta da due campi, nel primo specifichiamo un registro che contiene un valore, nel secondo campo inseriamo un valore. La somma dei due campi identifica l'indirizzo in memoria dell'operando. Ci sono varie tipologie di indirizzamento con offset:

indirizzamento con registro base:

```
1  LDR R1, [R0, #0xBB]
```

indirizzamento autorelativo:

```
1  LDR R0, [PC, #4]
```

aggiunge un offset al *program counter*. Viene utilizzato per i loop, if, procedure e altro.

1.3 Istruzioni in ARM

Per la lista completa delle istruzioni bisogna consultare l'ARM Quick Reference Card.

1.3.1 Istruzioni condizionate

Di default solo le istruzioni di confronto (come CMP) modificano sempre i bit **NZCV**, questi bit si trovano nel registro **CPSR**. Una generica istruzione modifica i *bit di stato* solo se si aggiunge il suffisso **S** al simbolo mnemonico dell'istruzione. Per esempio:

```
1 ADD R2, R0, R1 @ non modifica i bit NZCV
2 ADDS R2, R0, R1 @ modifica i bit NZCV
```

In particolare, in ARM, tutte le istruzioni possono essere condizionate e dunque modificare i bit del registro CPSR. Per esempio:

1.3.2 Sintassi delle istruzioni

Vediamo 3 classi di istruzioni che hanno una sintassi simile:

- **classe 1:** per istruzioni di elaborazione dati (come ADD, CMP, MOV, ...). In base all'istruzione, si possono essere a 3 indirizzi

```
1 <opcode>{<cond>}{S} <Rd>, <Rn>, <shifter_operand>
```

oppure a 2 indirizzi

```
1 <opcode>{<cond>}{S} <Rd>, <shifter_operand>
```

- **<opcode>**: codice identificativo dell'istruzione;
- **<cond>**: stabilisce l'esecuzione condizionata;
- **S**: stabilisce se modifica i bit di stato;
- **<Rd>**: destinazione, viene sempre specificato per primo;
- **<Rn>**: primo operando;
- **<shifter_operand>**: secondo operando. Può essere un valore immediato (**#valore**), un registro (**Rm**), oppure un registro con shift.

- **classi 2 e 3:** la classe 2 viene usata per Load&Store di word o unsigned byte, mentre la classe 3 viene usata per Load&Store di halfword o signed byte. In particolare usiamo

- **LDR:** carica nel registro di destinazione la word della locazione di memoria indicata dal metodo di indirizzamento. Per esempio:

```
1 LDR R0, [R1] @ carica in R0 M32[R1]
```

- **STR:** la word nel registro sorgente viene salvata nella locazione di memoria indicata dal metodo di indirizzamento. Per esempio:

```
1 STR R0, [R1] @ carica in M32[R1] il valore di in R0
```

di default leggono e scrivono word. Possiamo aggiungere dei suffissi B, H per leggere o scrivere byte o halfword, inoltre con i suffissi SB, SH possiamo scrivere byte, halfword con segno (non si usano per STR in quanto non ha senso estendere il segno in memoria).

Classe 2:

```
1 LDR|STR{B} Rd, <addressing_mode2>
```

dove **<addressing_mode2>** è un indirizzamento di registro *indiretto* con eventuale:

- senza offset (indirizzamento indiretto di registro):

```
1 LDR R1, [R0]
```

– offset immediato:

```
1 LDR R1, [R0, #4]
```

– offset da registro:

```
1 STR R2, [R0, R1]
```

– offset da registro scalato:

```
1 LDR PC, [PC, R0, LSL #2]
```

– pre-incremento immediato:

```
1 STR R5, [R0, #4]!
```

– pre-incremento da registro:

```
1 LDR R2, [R0, R1]!
```

– pre-incremento da registro scalato:

```
1 LDR R4, [R0, R1, LSL #2]!
```

– post-incremento immediato:

```
1 STR R1, [R0], #-8
```

– post-incremento da registro:

```
1 LDR R2, [R0], R1
```

– post-incremento da registro scalato:

```
1 LDR R4, [R0], R1, ASR #2
```

Classe 3:

```
1 STR|LDR[H] Rd, <addressing_mode3>
2 LDR[SH|SB] Rd, <addressing_mode3>
```

dove <addressing_mode2> è un indirizzamento di un registro con un eventuale:

- offset immediato (solo 8 bit);
- offset da registro;
- pre-incremento immediato;
- pre-incremento da registro;
- post-incremento immediato;
- post-incremento da registro.

Rispetto alla classe 2: non si possono scalare i registri e gli offset immediati sono solo a 8 bit.

1.3.3 Miscellanea

In ARM non esiste un'istruzione per caricare in un registro un valore immediato o un indirizzo immediato. Per ovviare il problema possiamo caricare un valore immediato valido con MOV e ottenere il valore voluto con somme e shift, oppure usiamo la **pseudo istruzione** come segue:

```
1 LDR R0, =0x45A0
```

l'assemblatore espande la *pseudo istruzione* in una o più istruzioni macchina.

2 Assembly in ARM

2.1 Struttura di un programma assembly

I programmi assembly sono costituiti da uno o più file testuali con estensione ".s". Il *programma eseguibile* viene costruito in codice assembly tramite il **compilatore** che si occupa dell'assemblaggio e del linkaggio. Per esempio, se vogliamo compilare il programma "sum.s" dobbiamo scrivere da riga di comando

```
1 gcc -o sum sum.s
```

Le *istruzioni assembly* non sono uguali alle *istruzioni macchina*, anche se c'è una corrispondenza quasi perfetta. In particolare le istruzioni macchina sono una rappresentazione binaria delle istruzioni assembly.

In un programma assembly troviamo:

- **simboli**: stringhe che denotano particolari valori come indirizzi in memoria, costanti, e alto. Si utilizzano spesso le **label**, un particolare simbolo che indica l'indirizzo in memoria di una particolare istruzione;
- **direttive**: ci permettono di *definire* simboli, *allocare* aree di memoria per salvare dati oppure *inizializzare* il contenuto delle aree di memoria.

Per allocare i dati possiamo usare le seguenti *direttive*:

- **.word**, **.hword**, **.byte**: per allocare e inizializzare in memoria rispettivamente 4 byte, 2 byte, 1 byte;
- **.ascii** per allocare e inizializzare una stringa;
- **.space**, **.skip**: per allocare aree di memoria (.space inizializza a 0).

Nei programmi assembly inoltre dobbiamo *segmentare il codice* in tre parti:

- **.text**: segmento che contiene le istruzioni;
- **.data**: segmento che contiene dati inizializzati;
- **.bss**: segmento che contiene dati non inizializzati

Analizziamo un esempio:

```
1 .text
2 .global main
3 main: LDR R0, =iterazioni
4       LDR R0, [r0]      @ numero iterazioni
5       MOV R1, #0        @ registro per somme parziali
6
7       ciclo: ADD R1, R1, R0
8              SUBS R0, R0, #1
9              BNE ciclo
10             LDR R2, =output
11             STR R1, [R2]    @ salva soluzione in memoria
12
13 .bss
14 output: .space 4
15 .data
16 iterazioni: .word 0x100
```

"main", "ciclo", "output", "iterazioni" sono tutte *label*, mentre ".space" 4 e ".iterazioni" sono *direttive*.

Nota che **ogni programma eseguibile deve avere un punto di inizio**, nell'esempio troviamo ".global main" che indica che il programma deve partire dalla *label* "main".

2.1.1 Costrutti di base

Listing 1: if-then-else

```

1  .data
2  addr_a: .word 10      @ inizializza a=10
3  addr_b: .word 20      @ inizializza b=20
4
5  .bss
6  addr_c: .skip 4       @ alloca c (4 byte)
7
8  .text
9  LDR R0, =addr_a       @ carica indirizzo di a
10 LDR R1, [R0]           @ copia a in R1
11 LDR R0, =addr_b       @ carica indirizzo di b
12 LDR R2, [R0]           @ copia b in R2
13 LDR R0, =addr_c       @ carica indirizzo di c
14 CMP R1, R2             @ confronta R1 e R2
15 STRLT R1, [R0]         @ imposta c=a se a<b
16 STRGE R2, [R0]         @ imposta c=b se a>=b

```

Listing 2: for loop

```

1  .data
2  addr_n: .word 100      @ inizializza n=100
3
4  .bss
5  add_sum: .skip 4       @ alloca sum (4 byte)
6
7  .text
8  LDR R3, = addr_n       @ copia indirizzo di n
9  LDR R2, [R3] @ copia n in R2
10 MOV R1, #0             @ R1 contiene somma parziale
11 MOV R0, #0             @ R0 e' il contatore i
12
13 for_loop: ADD R1, R1, R0 @ calcola sum=sum+i
14           ADD R0, R0, #1 @ incremento contatore
15           CMP R0, R2     @ confronta se i<=n
16           BLE for_loop   @ se si, ripeti loop
17           LDR R3, =addr_sum
18           STR R1, [R3]

```

2.2 Strutture dati

2.2.1 Array

Per definire un array bisogna specificare:

- l'indirizzo iniziale in memoria V;
- il numero degli elementi N;
- la lunghezza L di ciascun elemento (espressa in byte).

Nei programmi definiamo un array nel seguente modo

```

1  .bss
2  V: .skip N*L

```

dove "V" è una label che identifica l'indirizzo iniziale dell'area di memoria e il prodotto NL definisce l'estensione in byte.

Per accedere ai vari elementi si utilizza un metodo di **indirizzamento a due componenti**:

- una **componente fissa** è costituita dall'*indirizzo iniziale* V;
- una **componente variabile** che fornisce l'*offset dell'elemento* rispetto a V.

In generale conviene definire un array che parte da indice 0 in quanto ci risparmiano operazioni. Se l'indice parte da zero, l'indirizzo dell'elemento $V[i]$ (*ind* $V[i]$) è dato da

$$\begin{aligned} \text{Ind } V[i] &= V + \text{offset} \\ \text{offset} &= i \cdot L \end{aligned}$$

Se la lunghezza L è potenza di 2 allora il calcolo dell'offset può essere ottenuto mediante uno shift verso sinistra. Per esempio, se vogliamo copiare in R3 il valore di $V[i]$ e abbiamo $L = 4$ (vettore di word):

```

1  LDR R0, =V           @ indirizzo di V in R0
2  MOV R1, #i           @ indice i in R1
3  LSL R1, R1, #2        @ Calcola offset i*4
4  LDR R3, [R0, R1]     @ Copia V[i] in R3

```

Se la lunghezza L non è potenza di 2 allora siamo costretti ad utilizzare MUL per calcolare il prodotto $i \cdot L$. Nota che una moltiplicazione, in termini di tempo, è più costosa rispetto ad una somma o uno shift. Il codice diventa:

```

1  LDR R0, =V           @ indirizzo di V in R0
2  MOV R1, #i           @ indice i in R1
3  MOV R2, #L           @ parametro L in R2
4  MUL R1, R1, R2        @ Calcola offset i*L
5  LDR R3, [R0, R1]     @ Copia V[i] in R3

```

2.2.2 Matrici

Gli elementi di una matrice possono essere collocati in memoria ordinati per riga o per colonna. In generale per definire una matrice ci serve:

- l'**indirizzo iniziale** M;
- il **numero di righe** R;
- il **numero di colonne** C;
- la **lunghezza L di ciascun elemento**.

Nei programmi definiamo una matrice nel seguente modo

```

1  .bss
2  M: .skip R*C*L

```

Per individuare un elemento si usano gli indici di riga (i) e colonna (j). In generale l'indirizzo dell'elemento $M[i, j]$ è dato da

$$\text{Ind } M[i, j] = M + (i \cdot C + j) \cdot L$$

Anche in questo caso conviene usare un metodo di indirizzamento a due componenti:

- **componente fissa** ovvero *indirizzo base* costituito da M;
- **componente variabile** che fornisce l'*offset* calcolato valutando l'espressione $(i \cdot C + j) \cdot L$.

Vediamo un esempio: copiamo R0 in $M[i, j]$, poniamo che i sia contenuto in R1, j in R2, M in R4 e poniamo $L=2$

```

1  LDR R3, =C           @ Copia C in R3
2  MUL R3, R1, R3        @ Calcola R3 = i*C
3  ADD R3, R3, R2        @ Calcola R3 = i*C+j
4  LSL R3, R3, #1        @ Calcola R3 = (i*C+j)*2
5  STRH R0, [R4, R3]     @ Copia R0 in M[i,j]

```

Possiamo evitare il calcolo del prodotto $i \cdot C$ utilizzando un **array ausiliario di puntatori**, in questo modo l'accesso alla matrice si riconduce a 2 accessi ad array.

Definiamo la matrice M e l'*array ausiliario* P:

```

1  .bss
2  M: .skip R*C*L
3  .data
4  P: .word M, M+C*L, M+2*C*L, ..., M+(R-1)*C*L

```

Per esempio, con $L=2$, i in R1, j in R2, P in R4, trasferiamo R0 in $M[i, j]$:

```

1  LSL R3, R1, #2      @ i*4 (P contiene indirizzi)
2  LDR R3, [R4, R3]    @ Indirizzo riga i in R3
3  ADD R3, R3, R2, LSL #1 @ Indirizzo di M[i,j]
4  STRH R0, [R3]       @ Copia R0 in M[i,j]

```

Al posto dell'*array di puntatori* P, si può usare un **array ausiliario O con gli offset di ciascuna riga** rispetto all'indirizzo iniziale M.

Definiamo la matrice M e l'*array ausiliario* O:

```

1  .bss
2  M: .skip R*C*L
3  .data
4  O: .hword 0, C*L, 2*C*L, ..., (R-1)*C*L

```

rispetto ai *puntatori dell'array* P che sono di 4 byte, gli *offset* di O possono spesso essere da 2 o 1 byte. Inoltre un unico *array O di offset* può essere utilizzato per tutte le matrici che hanno lo stesso numero di righe, colonne e che hanno la stessa lunghezza degli elementi.

Per esempio, usando l'*array O di offset*, con $L=2$, i in R1, j in R2, M in R3, O in R4, trasferiamo R0 in $M[i, j]$:

```

1  LSL R5, R1, #1      @ i*2 (O contiene hwords)
2  LDRH R4, [R4, R5]   @ Offset riga i in R4
3  ADD R4, R4, R2, LSL #1 @ Offset di M[i,j]
4  STRH R0, [R3, R4]   @ Copia R0 in M[i,j]

```

2.2.3 Stack

Le operazioni che caratterizzano uno stack sono:

- **push**: aggiunge un nuovo elemento sopra agli altri già presenti;
- **pop**: toglie l'elemento in cima.

entrambe sono **pseudo istruzioni**, in ARM vengono tradotte come *store* e *load* multipli.

Ad ogni stack è associato un *puntatore* che punta alla **testa**, questo puntatore viene chiamato **stack pointer**. In generale ogni programma ha accesso ad uno stack di sistema, lo *stack pointer* si trova nel **registro SP** che è il registro R13.

Esistono varie implementazioni di stack, in questo corso vediamo uno stack **full descending**, ovvero uno stack che decresce con push e SP punta all'ultimo elemento usato.

Definizione di un'area di memoria di 4KB riservata ad uno stack (full descending):

```

1  .equ STL, 4096
2  .skip STL
3  stack: .skip 4

```

Inizializzazione dello stack pointer SP:

```

1  LDR SP, =stack

```

Per effettuare le operazioni di *push* e *pop*:

```

1  PUSH {R0} @ SP=SP-4 M[SP]=R0
2  POP {R0} @ R0=M[SP] SP+4

```

Possiamo effettuare *push* e *pop* **multipli**:

```

1  PUSH {R0-R7} @ vengono salvati nell'ordine R7,...,R0
2  POP {R0-R7} @ vengono salvati nell'ordine R0,...,R7

```

2.2.4 Liste concatenate

Ad ogni elemento di una lista concatenata è associato un *puntatore* che individua l'elemento successivo. Gli elementi contengono un **campo puntatore** (4 byte) e possono trovarsi ovunque in memoria. Anche nelle liste si sfrutta il puntatore **head** che punta al primo elemento oppure a **NIL** che è un valore di default (-1), indica che la lista è vuota.

Inizializzazione di una lista concatenata vuota:

```

1  .equ NIL, -1      @ -1 indirizzo di default
2  HEAD: .word NIL   @ lista vuota

```

Se, per esempio, R1 punta ad un elemento, con l'istruzione

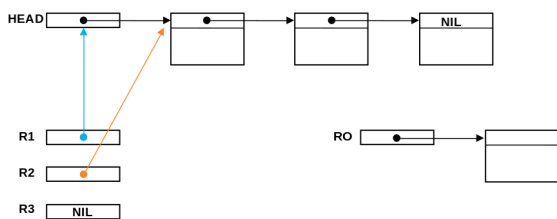
```

1  LDR R1, [R1]      @ leggo il puntatore di R1

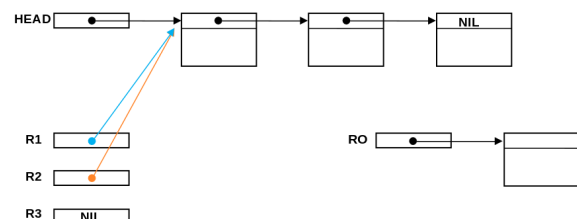
```

R1 punta all'elemento successivo. In questo modo si percorre la lista.

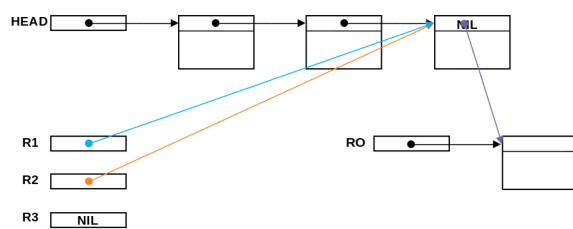
Inserzione di un elemento alla fine:



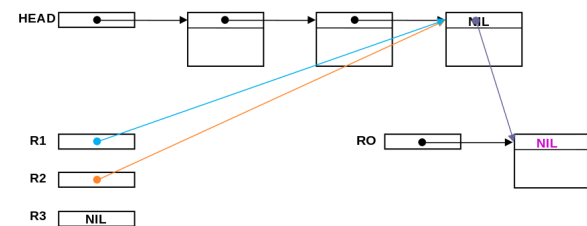
(a) 1 → 5



(b) 1 → 7



(c) 1 → 8

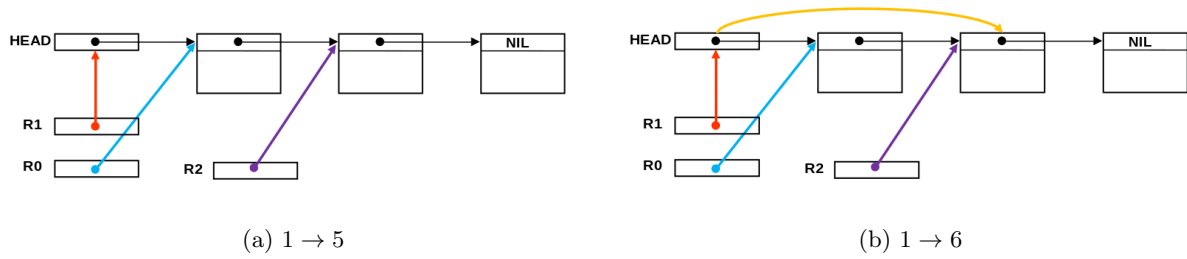


(d) 1 → 9

```

1  LDR R1, =HEAD      @ Puntatore iniziale
2  MOV R3, #NIL       @ Copio NIL in R3
3  CERCA: LDR R2, [R1] @ Carico indirizzo in R2
4  CMP R2, #NIL       @ E' ultimo elemento (R2=NIL)?
5  BEQ ULTIMO        @ Se ultimo salta a ULTIMO
6  MOV R1, R2         @ Altrimenti passa al prossimo
7  B CERCA           @ Ripeti
8  ULTIMO: STR R0, [R1] @ Aggancio
9  STR R3, [R0]       @ Ultimo elemento punta a NIL

```

Estrazione del primo elemento:

```

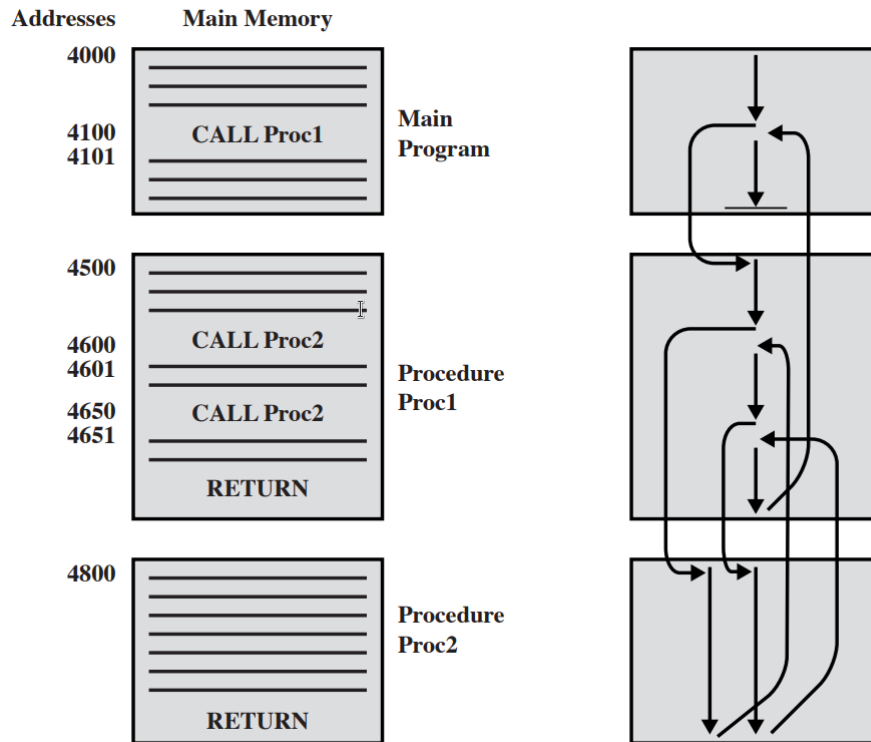
1  LDR R1, =HEAD    @ Puntatore iniziale
2  LDR R0, [R1]     @ Puntatore al primo elemento
3  CMP R0, #NIL     @ Lista vuota?
4  BEQ VUOTA        @ Si, non estrae
5  LDR R2, [R0]     @ Indirizzo prossimo elemento
6  STR R2, [R1]     @ Inserisci in HEAD
7  VUOTA: ...

```

3 Subroutine e funzioni

Una **procedura** o **subroutine** è una sequenza di istruzioni che può essere invocata da un altro programma. Le procedure vengono possono essere invocate in più parti del programma, il processore esegue il codice poi ritorna al punto da cui è partita la chiamata.

Il meccanismo delle procedure, per funzionare, richiede un'istruzione per chiamare la procedura e un'istruzione per ritornare al punto da cui è partita la chiamata (cioè l'istruzione successiva alla chiamata).



3.1 Definire una funzione

Una *funzione* viene definita in modo univoco dall'indirizzo di memoria contenente la prima istruzione. In assembly sfruttiamo le *label* per identificare l'inizio di una *funzione*, mentre per saltare da una label all'altra usiamo i salti B e BL.

Il ritorno da una funzione può essere implementato in vari modi:

1. salviamo l'*indirizzo di ritorno* in una **locazione di memoria adibita a tale scopo**, per esempio quella situata all'indirizzo 0.

Il problema di questo approccio è che se la procedura invocasse a sua volta un'altra procedura, il primo indirizzo di ritorno andrebbe sovrascritto e quindi perso.

```

1      MOV R0, #0      @ Indirizzo dove salvare PC
2      LDR R1, =RIT    @ Indirizzo di ritorno
3      STR R1, [R0]    @ Salva RIT in M[0]
4      B PROC          @ Salta alla procedura
5  RIT: ...            @ Prima istruzione da eseguire dopo il ritorno
6  PROC: ...
7      MOV R0, #0
8      LDR R0, [R0]
9      MOV PC, R0
  
```

2. l'indirizzo di ritorno viene salvato nelle **prima locazione di ciascuna subroutine**, con la convenzione che le istruzioni eseguibili della subroutine siano collocate a partire dalla locazione successiva

Il problema, in questo caso, sta nel fatto che non possiamo sfruttare la ricorsione.

```

1      LDR R0, =PROC    @ Indirizzo dove salvare RIT
2      LDR R1, =RIT     @ Indirizzo di ritorno
3      STR R1, [R0]     @ Salva il RIT in M[PROC]
4      B PROC+4         @ PROC+4 = PC
5
6      RIT: ...
7      PROC: .space 4   @ Word per l'indirizzo di ritorno
8      ...              @ Codice della funzione
9      LDR R0, =PROC
10     LDR R0, [R0]
11     MOV PC, R0

```

3. L'indirizzo di ritorno viene salvato in un **registro di CPU** anziché in una locazione di memoria.

Limitazione: questa soluzione consente alla subroutine PROC di chiamarne un'altra, purché venga usato un registro diverso.

```

1      LDR R14, =RIT    @ Indirizzo di ritorno
2      B PROC @ PROC = PC
3
4      RIT: ...
5      PROC: ...        @ Codice della funzione
6      ...
7      MOV PC, R14

```

4. L'indirizzo di ritorno viene memorizzato in uno **stack con un'operazione di push**.

Sfruttando questa implementazione, le subroutine possono essere **rientranti**, cioè di esse possono essere in esecuzione contemporaneamente più istanze. In questo modo possiamo sfruttare la ricorsione.

```

1      LDR R0, =RIT
2      PUSH {R0}        @ l'indirizzo RIT viene salvato nello stack
3      B PROC
4
5      RIT: ...
6      PROC: ...
7      ...
8      POP {PC}         @ Carica in PC il valore nello stack (ovvero RIT)

```

In ARM si usa un mix tra la terza e quarta soluzione: per invocare una procedura PROC si salva il punto di ritorno nel registro R14 chiamato **Link Register (LR)**, se necessitiamo di invocare altre procedure all'interno di PROC, basta salvare il valore di LR nello *stack*.

Per chiamare una procedura si usa l'istruzione **BL (Branch-and-link)**, essa salva l'indirizzo dell'istruzione successiva a BL in R14 (*indirizzo di ritorno*) e carica nel PC l'indirizzo della subroutine (*effettua il salto*).

In ARM non esiste un'istruzione apposita per il ritorno dalla subroutine, bisogna usare l'istruzione

```

1      MOV PC, LR @ torna dalla chiamata

```

3.2 Passaggio di parametri

Distinguiamo i parametri in **parametri di ingresso** (*dati passati alle subroutine*) e **parametri di uscita** (*risultati restituiti dalla subroutine*).

Il metodo più semplice e rapido per effettuare il passaggio di parametri consiste nello sfruttare i *registri* di CPU. Per esempio:

- **passaggio per valore:** per esempio vogliamo calcolare il modulo della differenza tra due numeri interi in complemento a due. Supponiamo di utilizzare: R0 parametro di uscita, R1 e R2 come parametri di ingresso.

La funzione ABS:

```

1  ABS: SUBS R0, R1, R2    @ Calcola R1-R2
2      RSBMI R0, R1, R2   @ Se negativo, calcola R2-R1
3      MOV PC, LR         @ Ritorna

```

in questo caso la subroutine modifica solo il parametro di uscita R0, mentre i parametri di ingresso R1, R2 non vengono modificati.

Chiamata di ABS:

```

1  .data
2  OP1: .word 13    @ Primo operando
3  OP2: .word 9     @ Secondo operando
4
5  .bss
6  RIS: .skip 4     @ Risultato
7
8  .text
9  LDR R3, =OP1     @ Carica indirizzo OP1
10 LDR R1, [R3]      @ Carica OP1
11 LDR R3, =OP2     @ Carica indirizzo OP2
12 LDR R2, [R3]      @ Carica OP2
13 BL ABS           @ Chiama la procedura ABS
14 LDR R3, =RIS      @ Carica indirizzo RIS
15 STR R0, [R3]      @ Salva output

```

- **passaggio per indirizzo:** anziché i valori dei parametri, è possibile passare gli indirizzi delle locazioni di memoria ove quei valori sono contenuti. Facendo riferimento all'esempio precedente: ci serve l'indirizzo del parametro di uscita R0 e dei parametri di ingresso R1, R2.

La funzione ABS:

```

1  ABS: LDR R1, [R1]    @ Carica OP1
2      LDR R2, [R2]    @ Carica OP2
3      SUBS R3, R1, R2  @ Calcola R1-R2
4      RSBMI R3, R1, R2 @ Se negativo, calcola R2-R1
5      STR R3, [R0]     @ Salva output
6      MOV PC, LR      @ Ritorna

```

Chiamata di ABS:

```

1  .data
2  OP1: .word 13    @ Primo operando
3  OP2: .word 9     @ Secondo operando
4
5  .bss
6  RIS: .skip 4     @ Risultato
7
8  .text
9  LDR R1, =OP1     @ Carica indirizzo OP1
10 LDR R2, =OP2     @ Carica indirizzo OP2
11 LDR R0, =RIS      @ Carica indirizzo RIS
12 BL ABS           @ Chiama la procedura ABS

```

al termine dell'esecuzione delle procedura, i registri R1 e R2, non contengono più gli indirizzi assegnati ma contengono i valori.

Che differenze ci sono tra i due metodi?

- **parametri passati per indirizzo:** avendo a disposizione l'indirizzo, la subroutine può accedere al valore originario del parametro e modificarlo;
- **parametri passati per valore:** se alla subroutine viene passato il valore (cioè copia del valore) di un parametro, eventuali modifiche apportate a questo valore dalla subroutine non coinvolgono il valore originario.

In generale, conviene che, dopo l'esecuzione di una subroutine, risulti modificato solo ciò è esplicitamente previsto che la subroutine modifichi, cioè i parametri di uscita (risultati).

Se la subroutine modificasse anche i parametri di ingresso, potrebbe causare effetti collaterali indesiderati durante l'esecuzione.

Una subroutine può avere la necessità di utilizzare dei registri per collocarvi i suoi risultati intermedi, ma allo stesso tempo non vogliamo modificare ciò che non è esplicitamente previsto. In questi casi conviene usare lo **stack**, in particolare:

- *salviamo il contenuto* dei registri usati dalla subroutine (che non siano usati per i parametri di uscita), prima di modificarli;
- *ripristiniamo il contenuto* di questi registri, prima di ritornare al programma chiamante.

Per esempio:

```

1 main:    BL SUB1                @ chiamata
2          . . .
3 SUB1:    PUSH {LR, R0-R2}        @ push LR e i registri usati
4          . . .
5          LDR R2, [R1, #4]!       @ modifica dei registri
6          ADD R0, R0, R2          @ modifica altri registri
7          BL SUB2                @ chiamata a una subroutine annidata
8          . . .
9          POP {LR, R0-R2}        @ pop LR e i registri usati
10         MOV PC, LR             @ ritorno

```

notiamo come questo metodo viene sfruttato anche per le chiamate annidate di subroutine, infatti nel programma viene salvato nello stack anche il registro **link register LR**. Se non ci fosse stata la chiamata alla procedura SUB2 non serviva fare un PUSH di LR.

3.3 Allocazione dinamica

Una subroutine si dice **rientrante** se di essa può iniziare una nuova esecuzione mentre è ancora in corso una sua esecuzione precedente. La cosa può accadere in diversi casi:

- in seguito a una **chiamata ricorsiva**;
- in seguito a una **interruzione**;
- in un **sistema multiprocessore**, ovvero se la subroutine si trova in memoria condivisa, più di un processore può iniziarne la esecuzione.

Una *subroutine rientrante* non deve alterare i dati su cui opera ogni sua attivazione precedente non terminata. Quando questa proseguirà la sua esecuzione, deve trovare gli stessi valori che vi aveva lasciato. (Per capire il concetto basta pensare alla ricorsione).

Importante specificare che **i dati non fanno parte di una subroutine**, ma dell'istanza di esecuzione, in quanto le subroutine sono fatte solo di codice (**pure code**) che può essere compilato e ricompilato.

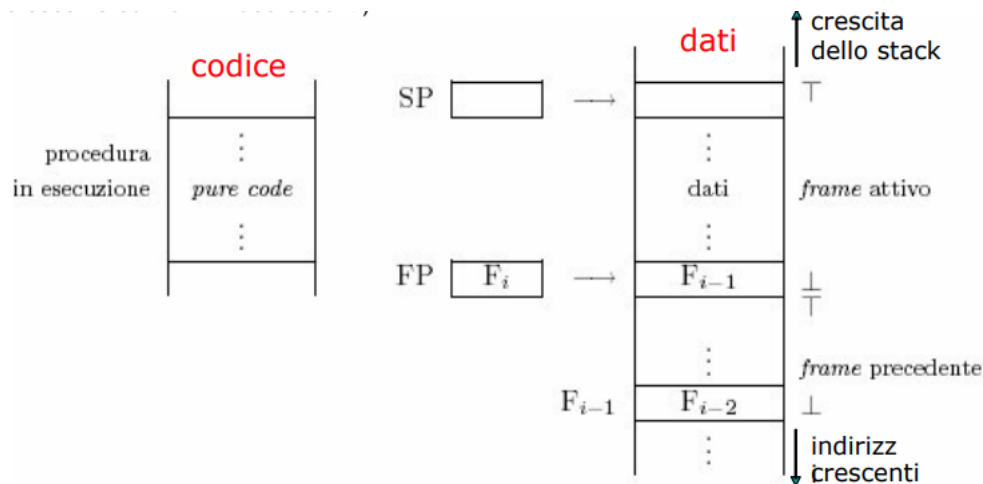
3.3.1 Subroutine rientranti

Per rendere una subroutine *rientrante* dobbiamo far sì che i dati su cui opera vengano allocati in un **luogo diverso** (per esempio nello *stack*). In questo modo potrà iniziare una nuova esecuzione mentre è ancora attiva quella precedente.

Più precisamente, i dati che devono essere memorizzati in ogni volta in un luogo diverso sono:

- i **parametri di ingresso** e di **uscita** passati alla subroutine (nota che anche l'indirizzo di ritorno memorizzato in LB è un parametro di ingresso);
- i **dati locali** su cui la subroutine opera.

Per implementare questo concetto si sfrutta l'**allocazione dinamica della memoria**. Praticamente, quando viene chiamata una subroutine, viene allocata in cima allo stack un'area chiamata **stack frame**, in cui si trovano tutti i dati (sia i *parametri* che i *dati locali*) sui quali la subroutine opererà. Infine, quando la procedura termina, questa area viene rimossa dallo stack.



Viene sfruttato il registro **Frame-Pointer FP** che punta all'inizio del frame, cioè all'indirizzo più alto dell'area di memoria da esso occupata (in quanto lo stack cresce verso indirizzi decrescenti). In questo modo, per accedere alle variabili, basta aggiungere un offset adeguato a FP.

3.3.2 Allocazione dinamica ARM e HLL

Listing 3: Esempio di subroutine ricorsiva (HLL)

```

1 void R(int I, int J, int *O) {  \ I, J par. ingresso (valore)
2                               \ *O par. uscita (indirizzo)
3     int A, B, C;  \ dati locali
4     ...
5     *O = I + J;
6     ...
7     if (A=0) R(A, B, &C);  \ chiamata ricorsiva
8     ...
9 }
10
11 int main() {
12     int W, X, Y, Z;  \ dati locali
13     ...
14     R(X, Y, Z);
15 }
```

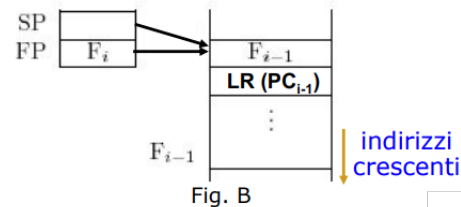
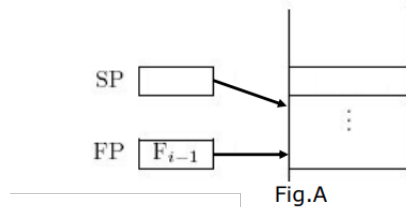
In assembly ARM:

Listing 4: inizializzo il frame

```

1 @ int main() {
2   main:           @ stack: Fig. A
3   PUSH {FP, LR}   @ FP, LR -> push
4   MOV FP, SP      @ SP -> FP
5                   @ stack: Fig. B

```



La *dichiarazione di una subroutine* si traduce in istruzioni che **allocano un nuovo frame** in cima allo stack.

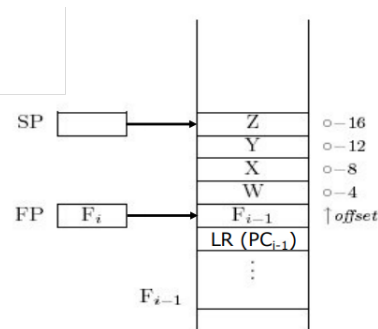
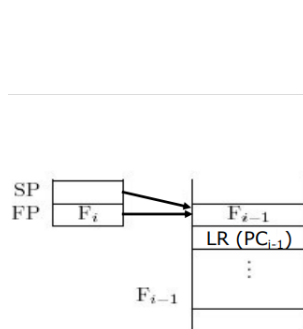
Notiamo che l'istruzione `PUSH {FP, LR}` ci permette di salvare i valori del *frame pointer* e del *link register*, in questo modo, quando la procedura avrà terminato l'esecuzione, saremo in grado di tornare alla situazione precedente alla chiamata (ovvero Fig. A).

Listing 5: dati locali nel frame

```

1 @ int W, X, Y, Z;
2                   @ stack: Fig. B
3   SUB SP, SP, #16 @ 4 interi di 4 byte = 16 byte
4                   @ W = FP-4
5                   @ X = FP-8
6                   @ Y = FP-12
7                   @ Z = FP-16
8                   @ stack: Fig. C

```



L'istruzione eseguita sottrae 16 allo *stack pointer*, in questo modo abbiamo lo spazio per allocare i dati locali. Per accedere ai vari dati si sfrutta l'indirizzo puntato dal *frame pointer* e lo si decrementa (o incrementa) di un certo offset.

Listing 6: chiamata alla subroutine R(...)

```

1 @ R(X, Y, &Z)
2                   @ stack: Fig. C
3                   @ inserzione parametri nello stack
4   LDR R0, [FP, #-8] @ X
5   PUSH {R0}         @ ->push
6   LDR R0, [FP, #-12] @ Y
7   PUSH {R0}         @ ->push

```

```

8  SUB R0, FP, #16    @ indirizzo di Z
9  PUSH {R0}          @ ->push
10                      @ chiamata alla subroutine
11                      @ stack: Fig. D
12  BL R              @ PC->LR, R->PC
13                      @ stack: Fig. D
14  ADD SP, SP, #12    @ rilascia l'area allocata per i parametri
15                      @ stack: Fig. C

```

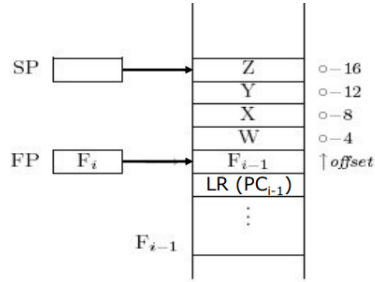


Fig. C

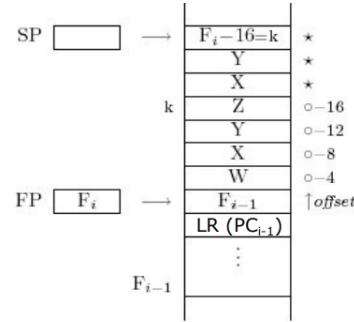


Fig. D

★ parametri
○ dati locali

In questa fase notiamo che i valori delle variabili X e Y (*parametri di ingresso*) vengono passati per valore mentre Z (*parametro d'uscita*) per indirizzo. La loro posizione nello stack è stata definita nella fase precedente.

Listing 7: rimozione del frame

```

1  @ }
2  MOV SP, FP    @ stack: Fig. C
3  POP {FP, PC} @ FP->SP il frame viene svuotato
4  @ stack: Fig. B
5  @ ripristina FP precedente
6  @ LR salvato->PC (ritorna al programma chiamante)
7  @ stack: Fig. A

```

Per rimuovere un frame dobbiamo sempre eseguire queste due istruzioni.

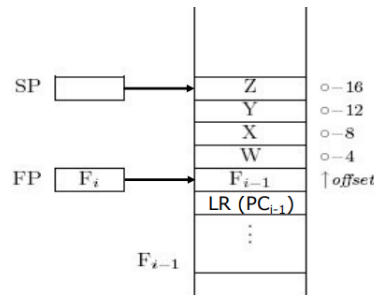


Fig. C

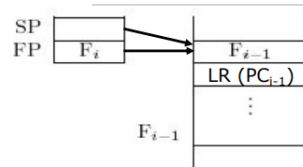


Fig. B

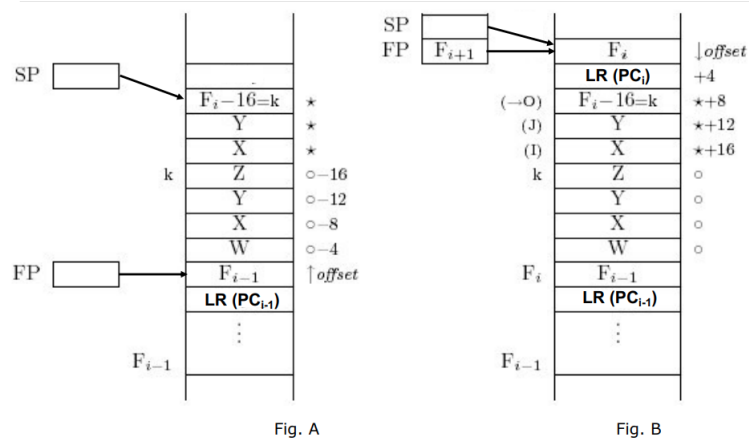
Listing 8: allocazione del nuovo frame con i parametri e dati locali della subroutine

```

1  @ void R(int I, int J, int *O) {
2  R: PUSH {FP, LR} @ stack: Fig. A
3  MOV FP, SP      @ FP, LR -> push
4  @ SP -> FP
5  @ I = FP+16
6  @ J = FP+12
7  @ -> O = FP+8
8  @ stack: Fig. B

```

Per allocare un frame, come prima cosa, dobbiamo sempre eseguire queste due istruzioni.

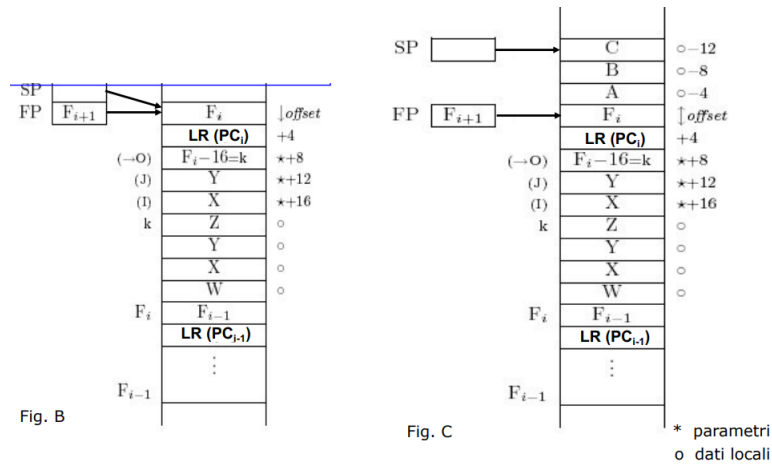


Listing 9: dati locali nel frame

```

1 @ int A, B, C;
2           @ stack: Fig. B
3   SUB SP, SP, #12 @ 3 interi di 4 byte = 12 byte
4           @ A = FP-4
5           @ B = FP-8
6           @ C = FP-12
7           @ stack: Fig. C

```



Listing 10: accesso ai parametri

```

1 @ *0 = I+J;
2           @ stack: Fig. C
3   LDR R0, [FP, #16] @ I
4   LDR R1, [FP, #12] @ J
5   ADD R0, R0, R1    @ I+J
6   LDR R1, [FP, #8]  @ ->0 (indirizzo di Z)
7   STR R0, [R1]      @ risultato in ->0, cioè' in Z
8           @ stack: Fig. C

```

Listing 11: chiamata ricorsiva

```

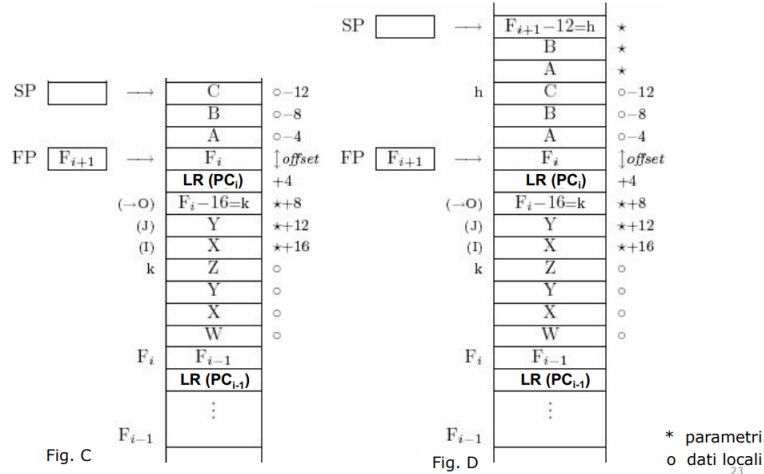
1 @ if (A==0)
2   LDR R0, [FP, #-4] @ il valore di A
3   CMP R0, #0       @ = 0 ?
4   BNE $1           @ indirizzo $1 generato dal compilatore
5 @ R(A, B, &C);
6   LDR R0, [FP, #-4] @ A
7   PUSH {R0}         @ ->push

```

```

8   LDR R0, [FP, #-8]    @ B
9   PUSH {R0}           @ ->push
10  SUB R0, FP, #12      @ indirizzo di C
11  PUSH {R0}           @ ->push
12                          @ stack: Fig. D
13  BL R                 @ PC->LR, R->PC
14                          @ stack: Fig. D
15  ADD SP, SP, #12      @ rilascia l'area allocata per i parametri
16                          @ stack: Fig. C

```



Listing 12: rimozione del frame

```

1  @ }
2  MOV SP, FP    @ FP->SP
3  POP {FP, PC} @ ripristina FP precedente
4                @ LR salvato->PC (ritorna al programma chiamante)

```

In generale, per quanto riguarda il **passaggio di parametri**, se sono **pochi** conviene inserirli nei *registri*, dopodiché è responsabilità della subroutine salvarli nello *stack* per poterli ripristinare o per essere rientrante; se sono **molti** se ne possono passare alcuni nei *registri*, altri nello *stack*.

Per esempio il compilatore in C: se i parametri sono ≤ 4 usa i registri R0, R1, R2 e R3. Altrimenti sfrutta i registri e lo stack. Ricordiamoci che sfruttando i registri risparmiamo letture in memoria e di conseguenza risparmiamo tempo.

3.4 Esempi di chiamate a subroutine in ARM

Per prima cosa precisiamo che la gestione software delle subroutine è basata su **convenzioni** arbitrarie. Queste devono essere rispettate dai programmatori: sia quelli che scrivono le subroutine sia quelli che scrivono i programmi che le usano.

Di conseguenza è fondamentale stabilire una convenzione sui parametri di ingresso e uscita (detta *interfaccia*). Una possibile *convenzione* è:

- i *parametri d'ingresso* vengono passati nei registri, a partire da R0;
- se i *parametri d'ingresso* sono troppi, si usa lo stack;
- i *parametri d'uscita* vengono passati nei registri, a partire da R0;
- se si vuol garantire la *rientranza* della subroutine, si usa la tecnica dello stack frame, dove bisogna inserire tutti i parametri e i dati locali;
- tutti i registri che non sono usati per i *parametri di uscita* devono rimanere inalterati, ovvero al termine della subroutine devono contenere gli stessi valori che avevano prima della sua chiamata.

Analizziamo un esempio:

- Lunghezza di una stringa:
vogliamo scrivere una subroutine che calcoli la lunghezza di una stringa. L'interfaccia della subroutine *strlen* è
 - *parametro di ingresso*: R0, puntatore alla stringa;
 - *parametro di uscita*: R1, lunghezza della stringa.

Listing 13: dati non inizializzati

```

1      .bss
2  lostack: .skip 2048      @ 2048 byte di stack "privato"
3  miostack:
```

dove l'indirizzo "*miostack*" indica il valore iniziale dello *stack pointer* SP (a stack vuoto).

Listing 14: dati inizializzati

```

1      .data
2  str1: .ascii "Questa stringa e' lunga 36 caratteri"
3      .byte 0 @ EOS (terminatore di stringa)
4      .align 4 @ allineamento a indirizzo di word
```

per calcolare la lunghezza della stringa basterà collocare in R0 l'indirizzo "*str1*" e chiamare la subroutine "*strlen*". Infine la lunghezza verrà restituita in R1.

Listing 15: inizializzazione

```

1      .text
2  main:
3      LDR SP, =miostack      @ inizializza SP
4      LDR R0, =str1          @ puntatore alla stringa str1
5      BL strlen              @ salta alla subroutine
6  pippo: B pippo             @ fine programma (=trappola)
```

abbiamo inizializzato il valore dello *stack pointer* SP all'indirizzo "*miostack*", inoltre abbiamo caricato in R0 l'indirizzo della stringa "*str1*".

Listing 16: subroutine *strlen*

```

1  strlen: PUSH {R2}          @ salva R2 sullo stack
2              @ R0: ind. inizio della stringa
3      MOV R1, #0             @ R1: lunghezza stringa
4  loop:   LDRB R2, [R0, R1]   @ carica in R2 il byte
5              @ di indirizzo R0+R1
6      CMP R2, #0             @ e' fine stringa (EOS)?
7      ADDNE R1, R1, #1        @ no: conteggia il carattere
8      BNE loop               @ esamina quello successivo
9  fine:   POP {R2}            @ ripristina i registri usati
10      MOV PC, LR            @ fine subroutine
```

la subroutine conteggia in R1 i caratteri fino as EOS escluso. In particolare viene caricato in R2 il carattere da esaminare.

Notiamo che R2 è l'unico registro che viene sovrascritto dalla subroutine, infatti la prima istruzione salva il contenuto nello stack e la penultima istruzione ripristina il suo stato; inoltre notiamo che non viene salvato il *link register* LR in quanto la subroutine non invoca altre procedure.

Lo stesso esempio poteva essere svolto sfruttando lo *stack frame*. In questo caso il codice del programma sarebbe stato:

Listing 17: *strlen*

```

1  strlen: PUSH {FP, LR}      @ salvo i registri FP, LR
2      MOV FP, SP            @ inizializzo il frame
3      PUSH {R0, R2}         @ salvo i registri usati
4      LDR R0, [FP, #8]       @ l'ultimo parametro ha sempre offset +8
```

```

5      MOV R1, #0
6 loop:  LDRB R2, [R0, R1]
7        CMP R2, #0
8        ADDNE R1, R1, #1
9        BNE loop
10 fine:  POP {R2}
11        MOV SP, FP          @ svuoto il frame
12        POP [FP, PC]        @ ritorno al chiamante

```

Listing 18: main

```

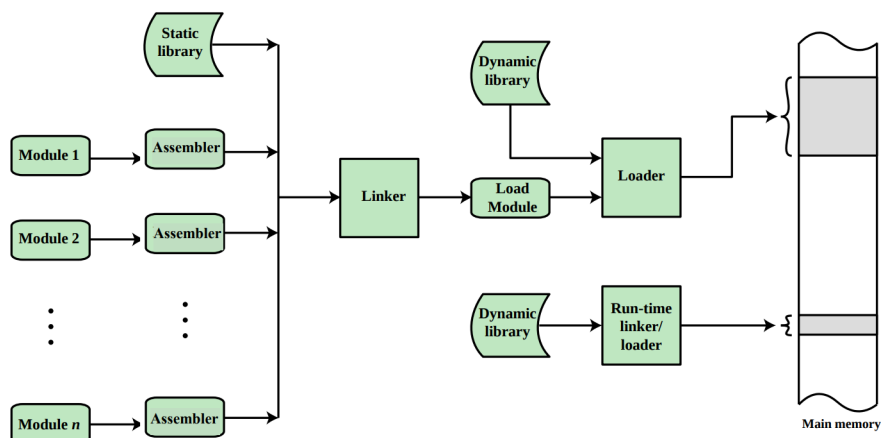
1 main:
2      LDR SP, =miostack
3      PUSH {FP, LR}        @ salvo i registri FP, LR
4      MOV FP, SP           @ inizializzo il frame
5      LDR R0, =str1
6      PUSH {R0}            @ passo l'indirizzo di str1
7      BL strlen
8      ADD SP, SP, #4       @ non ripristiniamo R0, cancelliamo il valore
9 pippo: B pippo

```

3.5 Costruzione di un file eseguibile

In assembly, la scrittura di un programma, necessita di cinque fondamentali elementi:

- **text editor:** un programma per la scrittura e la modifica del *testo sorgente*;
- **assemblatore:** un programma che traduce il *testo sorgente* in *modulo oggetto*;
- **linker:** un programma che costruisce il *programma eseguibile*;
- **loader:** un programma che carica in memoria il *programma eseguibile*;
- **debugger:** permette di eseguire il programma sotto il controllo del programmatore.



La struttura del codice è costituita da *moduli*, ognuno dei quali costituisce un *file sorgente* con estensione ".s". L'*assembler* opera in modo indipendente con ogni file sorgente e crea per ognuno un *file oggetto* con estensione ".o". Arrivati a questo punto il *linker* va a concatenare tutti i file oggetto e produce un singolo *file eseguibile*, infine il *loader* carica il file eseguibile in memoria e il programma viene eseguito.

3.5.1 Moduli sorgenti

In generale un **modulo sorgente** è costituito da *istruzioni macchina* e *istruzioni per l'assemblatore* (direttive e pseudo-istruzioni). Il codice di un programma consiste di uno o più moduli sorgente.

All'interno dei moduli utilizziamo i **simboli**, ovvero stringhe alfanumeriche che hanno un significato particolare definito dal linguaggio assembly oppure dal programmatore stesso. I simboli possono essere:

- simboli *opcode* (ADD, OR, ...), *registri*, *tecniche di indirizzamento*, ...;
- simboli che rappresentano *indirizzi di memoria* (tramite label);
- simboli che rappresentano valori numerici generici (tramite la direttiva .equ).

Inoltre il compilatore (o programmatore) ha il compito di classificare i simboli, in particolare possono essere:

- **simboli locali:** sono visibili solo nel modulo in cui vengono definiti. Non possono essere usati in altri file;
- **simboli globali:** sono visibili anche in altri moduli, possono essere usati in altri file;
- **simboli esterni:** vengono definiti in altri moduli sorgenti (in tale modulo sorgente devono essere globali).

I simboli hanno un valore che può essere: **assoluto** se non cambia, per esempio la direttiva .equ; **da rilocare** se dipende dalla posizione del codice in memoria, per esempio le label.

3.5.2 L'assembler

Oltre ai moduli oggetto, l'assemblatore segnala eventuali *errori* e *file di listing*. Il file di listing mostra come le istruzioni assembly sono state tradotte in istruzioni macchina.



L'assemblatore esegue **due scansioni** del file di input:

- **prima scansione:** cerca tutte le definizioni di simboli. Durante questa fase viene costruita una **tabella dei simboli** che contiene ogni simbolo definito nel sorgente. In particolare viene salvato il *nome* del simbolo, il suo *valore* e il *tipo* di simbolo (globale, locale).

Le operazioni eseguite sono:

1. leggi la prossima riga del sorgente;
2. se contiene la definizione di un simbolo X: inserisci il nome del simbolo, il suo valore, e se è globale/locale;
3. incrementa il **contatore LC** (*location counter*) del numero di byte necessari per codificare la riga letta. Il valore iniziale di LC è sempre 0, viene incrementato opportunamente ad ogni istruzione letta;
4. ritorna al passo 1.

Analizziamo un esempio:

LC=0	→	.TEXT		
LC=0	→	.global _start		
LC=0	→	_start: mov R1, #var_a		
LC=0x4	→	mov R0, #var_b		
LC=0x8	→	add R3, R1, R2		
LC=0xC	→	ldr R0, =ind_c		
LC=0x10	→	str R3, [R0]		
LC=0x14	→	trap: b trap		
LC=0x18	→	.equ var_a, 5		
LC=0x18	→	.equ var_b, 9		
LC=0x18	→	ind_c: .space 4		
LC=0x1C	→	.skip 0x100		
LC=0x11C	→	stack: .word 4		

Simbolo	Valore	Locale/ globale
_start		G
trap	0x14	L
var_a	5	L
var_b	9	L
ind_c	0x18	L
stack	0x11C	L

25

notiamo che le direttive e le dichiarazioni di costanti (`var_a`, `var_b`) non incrementano il counter LC in quanto non occupano memoria, inoltre notiamo che le ultime due istruzioni fanno incrementare LC dello spazio specificato da `.skip` e `.space`. Infine tutte le altre istruzioni incrementano il counter di 4 byte.

- **seconda scansione:** converte le istruzioni e risolve tutti i simboli.
 - Ogni simbolo presente nella tabella dei simboli viene sostituito con il suo valore e viene registrato l'indirizzo in cui è stato usato;
 - ogni istruzione assembly viene codificata in linguaggio macchina;
 - viene allocato lo spazio per i dati;
 - viene mantenuto il contatore LC.

Le operazioni eseguite sono:

1. leggi la prossima riga del sorgente;
2. sostituisci ogni simbolo presente nella tabella dei simboli con il suo valore; registra l'utilizzo dell'indirizzo;
3. codifica l'istruzione;
4. incrementa il contatore LC del numero di byte usati per codificare l'istruzione;
5. ritorna al passo 1.

Analizziamo un esempio:

LC=0	→	.TEXT	
LC=0	→	.global _start	
LC=0	→	_start: mov R1, #var_a	e3a01005
LC=0x4	→	mov R0, #var_b	e3a00009
LC=0x8	→	add R3, R1, R2	e0813002
LC=0xC	→	ldr R0, =ind_c	e59f0008
LC=0x10	→	str R3, [R0]	e5803000
LC=0x14	→	trap: b trap	ea000005
LC=0x18	→	.equ var_a, 5	
LC=0x18	→	.equ var_b, 9	
LC=0x18	→	ind_c: .space 4	00000000
LC=0x1C	→	.skip 0x100	0.....0
LC=0x1C	→		00000000
LC=0x1C	→	stack: .word 4	00000018

Simbolo	Valore	Locale/ globale	Indirizzi uso
_start	0x0	G	-
trap	0x14	L	0x14
var_a	5	L	0x0
var_b	9	L	0x4
ind_c	0x18	L	0x120
stack	0x11C	L	-

sostanzialmente vengono codificate le varie istruzioni seguendo i codici operativi. Per esempio prendendo la prima istruzione, notiamo che l'assembler riconosce il simbolo `var_a` e assegna correttamente il valore immediato (cioè 5). Notiamo inoltre che per riservare memoria a `ind_c` e allo `stack`, vengono messi tanti zeri a seconda dello spazio richiesto.

Se avessimo definito anche *simboli esterni* definiti in altri file sorgenti allora l'assembler avrebbe creato una **tabella dei simboli esterni**. In questa tabella vengono elencati i simboli mancanti e gli indirizzi nel modulo oggetto in cui il valore mancante deve essere inserito.

In questo caso sarà compito del *linker* di inserire i simboli mancanti.

Abbiamo già detto che l'*assembler* lavora indipendentemente con ogni file sorgente, inoltre specifichiamo che lavora indipendentemente anche su ogni segmento.

In questo modo: ogni segmento viene costruito in un proprio spazio di indirizzamento (quindi LC parte ogni volta da 0) e vengono assemblati indipendentemente (verranno ricomposti dal *linker* usando un unico spazio di indirizzamento); simboli dichiarati in segmenti diversi vengono gestiti come *simboli esterni* (quindi inseriti nella *tabella dei simboli esterni*).

Precisiamo che viene comunque generato un unico *file oggetto* ".o".

3.5.3 Il linker

Il *linker* unisce gli *N moduli oggetto* in un unico file eseguibile contenente: il programma in linguaggio macchina; informazioni di supporto (indirizzo di partenza per eseguire il file).

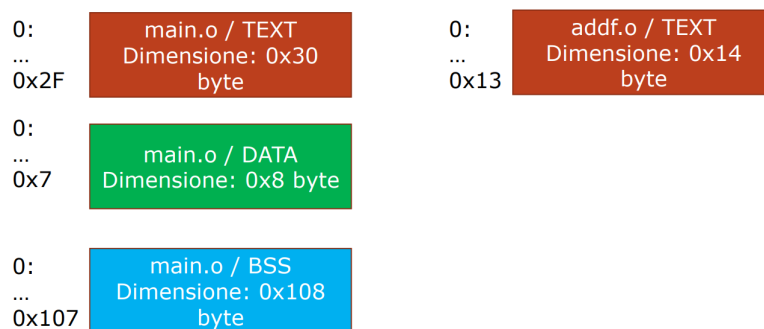


Come abbiamo anticipato, il *linker* sfrutta le tabelle per creare il *modulo eseguibile*, in particolare, genera i segmenti TEXT, DATA, e BSS dove vengono collocate le porzioni di quel segmento provenienti da ciascuno degli *N* moduli.

Le operazioni eseguite sono:

1. calcola l'estensione di memoria occupata da ciascun segmento di ciascun modulo.

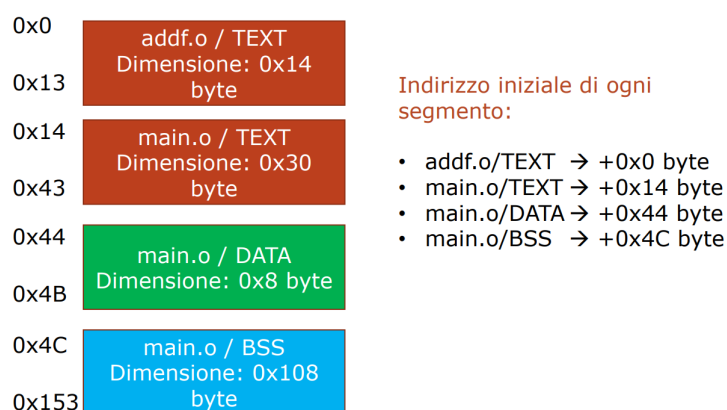
Per esempio:



(file: main.s; addf.s)

2. posiziona i segmenti in memoria e calcola il nuovo indirizzo iniziale (è necessario in quanto ogni segmento parte con il counter LC a zero).

Per esempio:



3. ogni indirizzo di memoria definito da una label viene riallocato. Gli indirizzi definiti tramite label in memoria devono essere aggiornati, perciò, per ogni label *X* con valore *V* all'interno di un segmento con indirizzo iniziale *ADR*, il *linker* sostituisce tutte le occorrenze di *X* con il valore *V+ADR*.

Per esempio: si consideri una label *X* con valore 0x105 all'interno di un segmento che avrà indirizzo iniziale 0xAA000. Il linker sostituirà tutte le occorrenze di 0x105 con il valore 0xAA000+0x105 = 0xAA105.

Importante specificare che non vengono riallocati:

- indirizzi o valori assoluti, ovvero definiti da costanti (.eqi);
 - indirizzi definiti da offset (autorelativo, frame pointer) in quanto dipendono dalla posizione del PC relativa a quando viene eseguito il programma.
4. tutti i riferimenti a simboli esterni vengono risolti. Ogni riferimento a simbolo esterno viene risolto con l'utilizzo della *tabella dei simboli esterni* e di tutte le tabelle dei simboli (tabelle aggiornate con la riallocazione).

3.5.4 Il loader

Il *loader* viene invocato per eseguire il programma. Il suo compito è quello di caricare in memoria le istruzioni macchina e dati del programma agli indirizzi indicati nel *file eseguibile* e carica nel registro PC l'indirizzo del punto di inizio (ovvero `_start`).

In alcuni casi il *loader* può riallocare il programma in nuovi indirizzi, questo processo viene eseguito come la riallocazione eseguita dal *linker*.

3.5.5 Le librerie

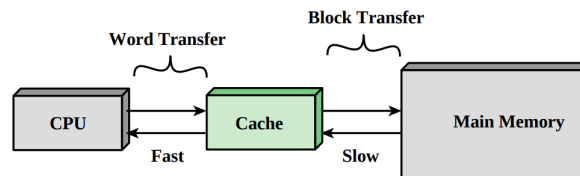
Le **librerie software** sono collezioni di moduli oggetto che contengono subroutine che possono essere invocate da altri programmi. Il codice presente nelle *librerie* può essere inserito in diversi modi, per questo differenziamo:

- **librerie statiche:** il codice di queste librerie viene incluso nel momento dell'*assemblaggio* e del *linking*. Il codice viene inserito nel *file eseguibile*, in questo modo, se qualcuno aggiorna la libreria, siamo costretti a ricompilarlo;
- **librerie dinamiche:** il codice delle librerie viene incluso in un momento successivo al linking (non si trovano nel *file eseguibile*), dunque il *linker* non risolve i simboli della libreria dinamica, restano indefiniti. Questi simboli vengono risolti quando il programma viene caricato in memoria e il loader inserisce quelli mancanti (*load-time dynamic library*), oppure quando il programma esegue l'istruzione con il simbolo mancante (run-time dynamic library), questo caso viene gestito dal sistema operativo.

4 Memorie cache, paging, memoria virtuale

Le principali caratteristiche di una memoria sono: dimensione, tempo di accesso e costo per bit. Per quanto riguarda la memoria RAM, la sua velocità di trasferimento è molto più lenta rispetto alla velocità di elaborazione dati della CPU. Ciò comporta dei rallentamenti in quanto il processore è costretto ad aspettare che i dati arrivino dalla RAM. Per questo introduciamo la *cache*, ci permetterà di velocizzare questo tipo di processi.

4.1 Gerarchie di memoria e principio di località di riferimento



La **memoria cache** si trova tra la CPU e la ram, essa ha caratteristiche intermedie, è più veloce della RAM ma ha un'estensione limitata. La cache contiene una copia dei dati presenti in memoria principale, in questo modo, quando il processore richiede l'accesso ad una word in memoria, controlla se tale word è presente nella *cache*, in caso negativo accede alla memoria.

Questi due processi hanno un nome preciso: se il processore trova la word che cercava nella cache allora si parla di **cache hit**; se la word non è presente nella cache allora abbiamo un **cache miss**, in questo caso il processore è costretto ad accedere alla memoria e quel dato, insieme ad un blocco di word consecutive, viene caricato nella cache.

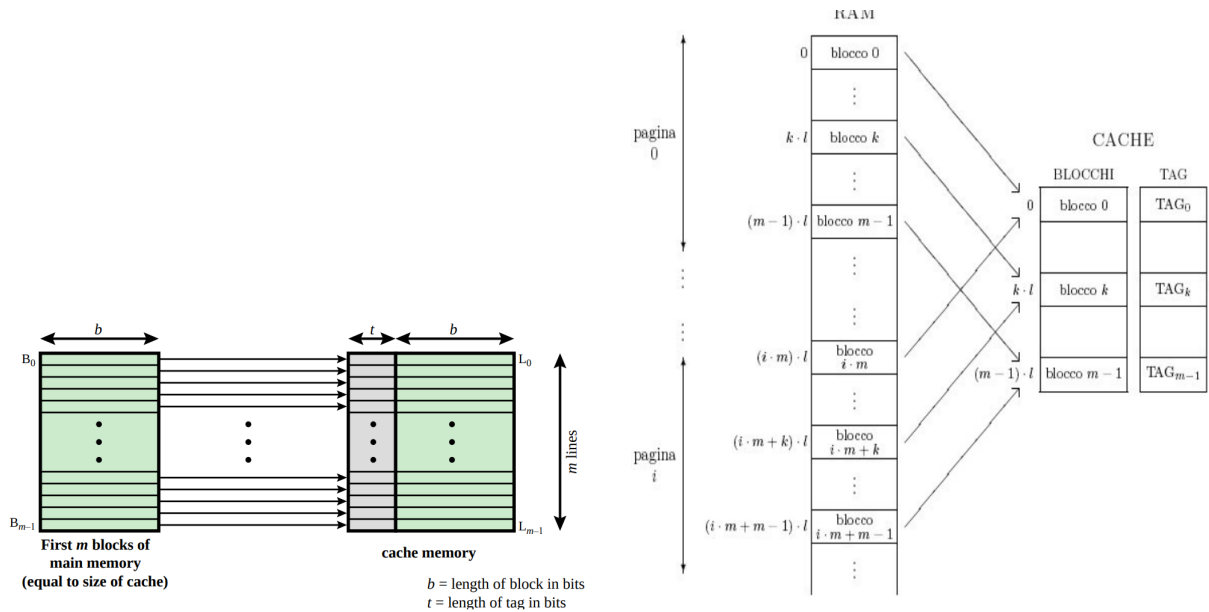
I dati nella cache vengono inseriti quando il processore li richiama spesso (**località temporale**), in questo modo i dati usati più spesso vengono mantenuti in cache quindi l'accesso risulta più veloce. I dati meno usati risiedono in memoria principale, il loro accesso costa di più, ma vengono richiesti raramente. Il problema è che la cache ha una dimensione limitata e inoltre non sappiamo quali dati verranno utilizzati più frequentemente nel futuro.

Per ovviare al problema la cache sfrutta anche la **località di riferimento spaziale**. Sostanzialmente, quando il processore è costretto ad accedere alla memoria principale, la cache copia i dati vicini a quello richiesto, copia un *blocco* di words, in questo modo aumenta la percentuale di *cache hit* nel tempo.

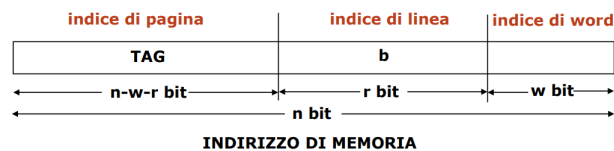
Per calcolare il costo medio di un accesso alla memoria si sfrutta la seguente formula

$$T = T_1 \cdot p + T_2(1 - p)$$

dove T_1 è il tempo per accedere ad un dato in cache; T_2 è il tempo per accedere ad un dato in memoria (si ha sempre $T_1 \ll T_2$); p è la percentuale di hit rate.



Per questa tipologia di cache gli indirizzi sono divisi in tre campi:



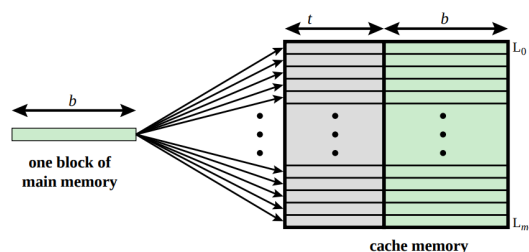
sostanzialmente gli s bit sono ora divisi in: gli r bit meno significativi indicano l'indice della linea di cache; i rimanenti $n - w - r$ bit indicano il tag, cioè l'indice della pagina della RAM.

La **cache a mappatura diretta** opera nel seguente modo: il processore genera un indirizzo di memoria; vengono prelevati gli r bit che identificano l'indirizzo di linea e viene cercata nella cache tale riga i -esima; successivamente vengono prelevati gli $n - w - r$ bit che identificano il tag; avviene un confronto tra il tag prelevato dall'indirizzo e il tag associato alla linea della cache individuata in precedenza. Se corrispondono allora abbiamo un *cache hit*, in caso contrario il blocco cercato si trova in un'altra pagina della RAM (*cache miss*), quindi viene recuperato e, il blocco di RAM e il suo tag, vengono inseriti nella linea della cache individuata in precedenza.

Questo tipo di cache è caratterizzata da un'organizzazione semplice, tuttavia ha dei problemi. Infatti le linee da sovrascrivere sono predeterminate: un blocco non può essere inserito in una linea scelta tra quelle che presumibilmente non servono più, ma viene sovrascritto sempre nella stessa posizione. Per esempio in un programma in cui dobbiamo accedere a due dati e gli indirizzi in memoria di questi vengono scritti nella stessa linea della cache, il dato non verrà mai trovato nella cache in quanto tale linea verrà modificata dopo ogni lettura. In questo caso il *cache miss* diventa del 100%. Per questo motivo la cache a mappatura diretta ha un *hit rate* basso quindi è poco efficiente.

4.3.2 Completamente associativa

Nella cache **completamente associativa**, un blocco viene inserito in una linea di cache vuota. Nel caso in cui le linee in cache siano tutte occupate, viene scelta quale linea svuotare con una *politica di rimpiazzo*.



Gli indirizzi vengono divisi in due campi (come a pagina 29).

La **cache completamente associativa** opera nel seguente modo: il processore genera un *indirizzo di memoria*; vengono prelevati dall'indirizzo i $n - w$ bit che identificano il *tag*; avviene il confronto in parallelo con tutti i tag delle m linee della cache. Se il tag viene trovato abbiamo un *cache hit* e l'accesso di risolve nella cache. In caso contrario (*cache miss*) significa che il blocco cercato appartiene ad un'altra pagina, di conseguenza viene recuperato con un accesso alla RAM e successivamente viene copiato insieme al suo *tag* in una linea vuota della cache.

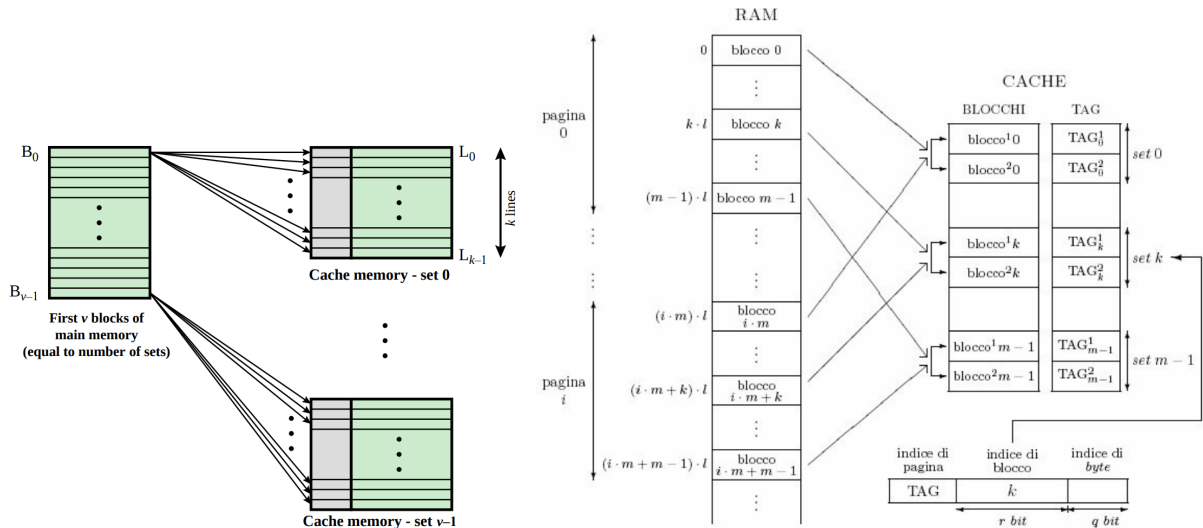
Per effettuare la ricerca del tag nella cache viene sfruttata una **memoria associativa** o **CAM** (Content Addressable Memory): confronta in parallelo il dato da cercare con tutti i dati in essa contenuti; se il dato è presente restituisce in output il suo indirizzo. Questo tipo di memoria riceve un dato e restituisce l'indirizzo (o gli indirizzi) delle celle che contengono quel dato, a differenza della RAM che riceve un indirizzo e restituisce il dato contenuto a quell'indirizzo.

La CAM sfruttata in questo tipo di cache è complessa e molto costosa in quanto si presuppone che sia molto veloce e molto grande (deve contenere tutti i tag).

La *cache completamente associativa* ha un *hit rate* più alto della cache a *mappatura diretta*, il problema è che è costosa a causa della CAM.

4.3.3 Set-associativa a k vie

La cache **associativa a k-vie** rappresenta un compromesso tra la due viste precedentemente. In generale è la più utilizzata.



Per ogni indice di blocco (*tag*) ci sono k linee nella cache invece che una. A ciascun *set* di k linee è associata una memoria CAM di k elementi che contiene i *tag* dei blocchi presenti nel *set*. Come nell'organizzazione a *mappatura diretta*, la RAM è ancora divisa in pagine e ciascuna pagina in blocchi. La differenza è che: l'indice di blocco individua il *set* in cui sarà caricato; il blocco verrà caricato all'interno del *set* in una qualsiasi delle delle k linee.

Nell'immagine sopra (a destra) viene riportato un esempio di *cache set-associativa a 2 vie*: il blocco 0 della memoria può essere inserito solo nel *set* 0 della cache; all'interno del *set* può essere inserito in una delle due righe indifferentemente (e così vale anche per tutti gli altri blocchi).

Il numero degli elementi della memoria CAM è uguale al numero delle k vie e per ogni *set* viene implementata una memoria. In questo modo ci sono molte memorie CAM che sono relativamente piccole, ognuna è indipendente dalle altre. Nella memoria *cache completamente associativa* abbiamo visto che veniva implementata un'unica CAM grande che gestiva tutti i tag dei blocchi e questo risultava un problema in termini di costi; introducendo tante CAM piccole smorziamo il problema.

Gli indirizzi in questo caso sono divisi in tre campi:



La **cache set-associativa a k vie** opera nel seguente modo: il processore genera un *indirizzo di memoria*; vengono letti gli r bit che identificano l'*indirizzo di set*; la memoria CAM prende come input i $n - w - r$ bi che identificano il *tag* ed esegue una ricerca in parallelo del tag all'interno del *set*. Se il tag è presente allora c'è un *cache hit* quindi viene letto l'indice di word composto da w bit e viene prelevato il byte corrispondente dal blocco. In caso contrario si ha un *cache miss*, il dato viene letto in memoria all'indirizzo richiesto, inoltre il blocco di RAM caricato viene trasferito in una delle linee del *set* e il suo *tag* nel corrispondente elemento della CAM.

4.4 Algoritmi di rimpiazzo

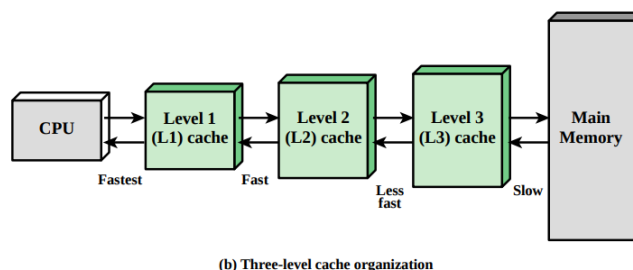
Se la memoria cache è piena, che dati rimuoviamo per far spazio? Esistono due approcci:

- **FIFO**: first in first out, viene sovrascritta la linea contenente il blocco che si trova nella cache da più tempo. Si guarda l'ordine;
- **LRU** least recently used, viene sovrascritta la linea che da più tempo non subisce accessi. Vengono sfruttati uno o più dei bit di controllo associati alla linea, per tener traccia del tempo trascorso dall'ultimo accesso.

Anche per quanto riguarda la scrittura dei dati esistono due tecniche:

- **copy-back**: Se un blocco in cache ha subito accessi per la scrittura (un bit di controllo, **dirty bit**, lo segnala) viene aggiornato anche in RAM. In particolare il blocco viene aggiornato in RAM solo se viene rimosso o sovrascritto nella cache. Questo metodo è più veloce in quanto i dati vengono copiati in RAM molte meno volte.
- **write-through**: cache e memoria principale sono sempre allineate. Tutte le operazioni di scrittura vengono effettuate sia nella cache sia nella RAM. Questo metodo è più lento in quanto richiede molti accessi in memoria.

4.5 Gerarchia multilivello



Possono essere presenti più **livelli di cache**:

- una cache di **primo livello**, quasi sempre integrata nello stesso chip del processore, ad accesso rapidissimo. Questa conterrà i dati che serviranno al processore nell'immediato futuro;
- una cache di **secondo livello**, talvolta esterna al chip del processore, ad accesso rapido. A volte anche una cache di **terzo livello**. Queste sono più lente ma più capienti, contengono i dati che serviranno più avanti.

4.6 Gestione della memoria

CPU e memoria sono risorse che possono essere condivise da più programmi. L'idea è quella di poter eseguire più programmi contemporaneamente: se un programma viene caricato in memoria e viene eseguito, non vogliamo che il processore si dedichi completamente solo a quell'unico programma dall'inizio della sua esecuzione fino alla fine. Immaginiamo un programma che deve ricevere dati in input, quindi ci sarà un tempo di attesa e magari in quel tempo dobbiamo far eseguire un altro programma che risponda con i dati in input.

Sfruttando un sistema **multi-programming** (o multi-tasking) otteniamo questo tipo di organizzazione della memoria, in particolare si ottiene un uso più efficiente della memoria e del processore.

Per consentire la gestione multi-tasking dei processi, un Sistema Operativo utilizza appositi meccanismi forniti dall'hardware che consentono di: **suddividere la memoria** per consentire in essa la presenza di più processi; suddividere la memoria in modo efficiente in modo tale da farci stare il maggior numero possibile di processi.

Le tecniche di gestione della memoria più utilizzate sono

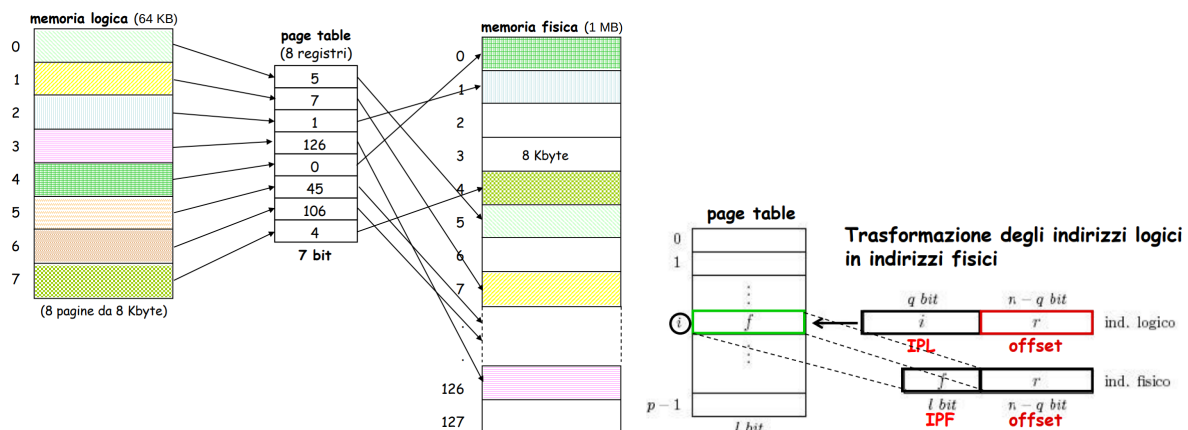
- **paginazione;**
- **segmentazione;**
- **paginazione con memoria virtuale;**
- **segmentazione con memoria virtuale.**

4.6.1 Paginazione

La memoria fisica viene suddivisa in blocchi chiamati **pagine fisiche** (o **frame**), tutte della stessa dimensione (spesso piccola). Stessa cosa vale per i processi: vengono divisi in blocchi delle medesime dimensioni, detti **pagine logiche** (o **pagine**).

La paginazione permette di mappare *pagine logiche* in *pagine fisiche*, per farlo viene sfruttato un meccanismo chiamato **MMU** (Mapping and Management Unit). L'MMU è un'unità esterna alla CPU, il suo compito è quello di tradurre gli indirizzi logici in uscita dal processore nei corrispondenti indirizzi della memoria fisica.

Analizziamo il seguente esempio, in cui la memoria logica è più piccola della memoria fisica:



La **page tabel** mappa gli indirizzi logici in indirizzi fisici, ad esempio vediamo che la pagina logica 0 è stata mappata con la pagina fisica 5, quindi non viene per forza rispettato l'ordine. Il fatto che le pagine non siano consecutive ci permette di portare in memoria più processi.

Per quanto riguarda gli indirizzi fisici e logici, essi vengono suddivisi in 2 campi. Quelli logici contengono: l'**IPL** che indica l'indice pagina logica; l'**offset** che dipende dalla dimensione delle pagine. Gli indirizzi fisici contengono: l'**IPF**, ovvero l'indice della pagina fisica; l'**offset**, identico a quello degli indirizzi logici (in quanto le pagine hanno dimensione uguale).

Nella page table troviamo la parte più significativa dell'indirizzo fisico (IPF) alla posizione i data dalla parte più significativa dell'indirizzo logico (IPL).

Nell'esempio riportato abbiamo un processore che genera indirizzi (logici) da $n = 16$ bit (può indirizzare $2^{16} = 64$ Kbyte); la memoria fisica è di 1 Mbyte (indirizzi fisici da $m = 20$ bit):

- si può suddividere lo spazio degli indirizzi logici in 8 pagine da $64K/8 = 8$ Kbyte, quindi l'indice di pagina logica è di 3 bit ($2^3 = 8$: $q = 3$) e 8 Kbyte corrispondono ad un offset di 13 bit;
- la memoria fisica risulta comprendere $1M/8K = 128$ pagine fisiche, l'indice di pagina fisica è quindi di 7 bit ($2^7 = 128$: $l = 7$);
- la MMU ha una page table con 8 registri di mappa (uno per ciascuna delle 8 pagine logiche) da 7 bit contenenti gli indici dei frame.

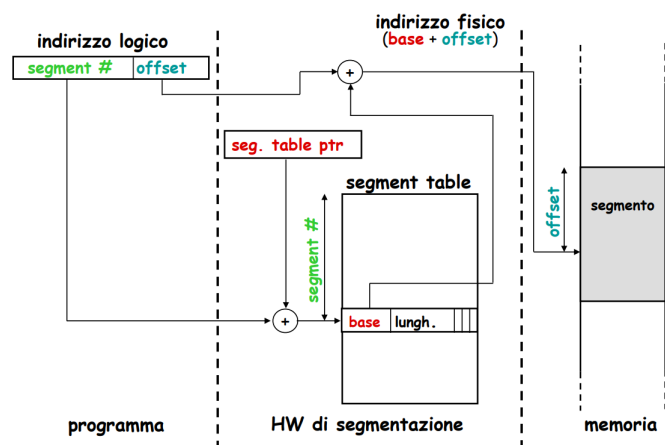
La MMU ha diversi vantaggi, tra cui: la protezione (bit di controllo per impostare pagine come read only e quindi controllare gli accessi), rilocamento, frammentazione dello spazio libero in memoria (quando un programma termina tutti i suoi processi ne vengono inseriti altri di altri programmi, ovunque essi si trovino), protezione da interferenze tra programmi e comunicazione tra processi.

Il problema sostanziale è che la dimensione delle pagine è fissa, in questo modo se dobbiamo eseguire un programma che ha dimensione poco più grande di una pagina questo andrà ad occupare 2 pagine nella memoria, di conseguenza c'è uno spreco.

4.6.2 Segmentazione

La segmentazione vuole risolvere i problemi della paginazione. I moduli (segmenti) di un processo possono essere di dimensione variabile. Gli indirizzi sono composti da 2 campi di dimensione variabile: un **indice di segmento** e un **offset**.

A ciascun processo viene associato un insieme di segmenti (uno per ciascuno dei suoi moduli) e una **segment table** contenente: l'indirizzo iniziale del segmento; la dimensione del segmento; bit di controllo; bit di protezione.



La trasformazione da *indirizzo logico* a *indirizzo fisico* avviene come segue:
la CPU emette l'indirizzo logico che ha due componenti: un **indice di segmento IS** e un **offset OS**. L'indice di segmento individua un elemento della *segment table* da cui si estrae l'indirizzo fisico iniziale del segmento (IIS) e la dimensione del segmento. Successivamente l'offset OS viene confrontato con la dimensione DS:

- se $OS \leq DS$ l'indirizzo è valido e allora l'**indirizzo fisico è dato da IIS + OS** (indirizzo iniziale del segmento, estratto dalla segment table, + l'offset, estratto dall'indirizzo logico);
- se $OS > DS$ l'indirizzo non è valido.

4.7 Memoria virtuale

Il problema della segmentazione è che la traduzione dell'indirizzo logico richiede tempo, è più lenta della paginazione. Per questo si sfrutta la **memoria virtuale**. Con questo metodo i programmi non serve che vengano caricati completamente in memoria per poter essere eseguiti, il sistema operativo può caricare in memoria solo i blocchi che servono in quel momento. In questo modo possiamo eseguire molti più programmi, la memoria virtuale può essere più grande della memoria fisica.

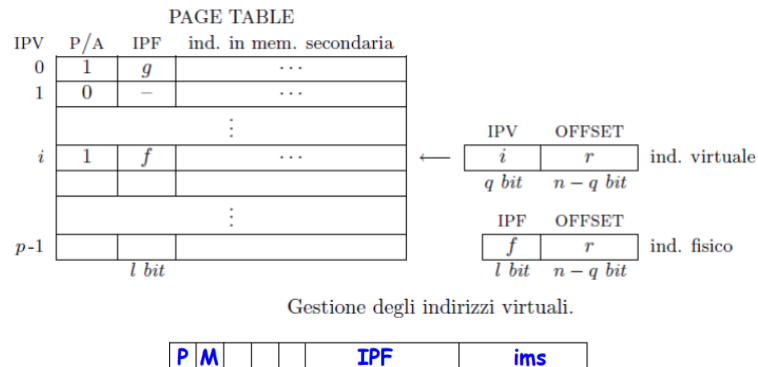
Il sistema di **memoria virtuale** ha due conseguenze importanti:

- **il numero di processi presenti in memoria è maggiore** (i programmi non vengono caricati completamente);
- **un singolo processo può indirizzare più memoria di quella fisicamente disponibile**, questo perché la memoria virtuale serve a simulare una memoria fisica più grande. Le pagine fisiche più importanti vengono messe nella RAM, quelle che non ci stanno vengono caricate sul disco.

Affinché un sistema operativo possa realizzare un sistema di memoria virtuale, è necessario che nell'hardware sia presente: un *meccanismo di paginazione*; un *meccanismo di segmentazione*; oppure entrambi.

Sfruttando questi meccanismi il sistema operativo può gestire i trasferimenti delle pagine e/o dei segmenti tra memoria centrale e disco; l'organizzazione delle *page table* e delle *segment table* va rivista per tener conto delle esigenze del sistema di memoria virtuale.

4.7.1 Paginazione e memoria virtuale



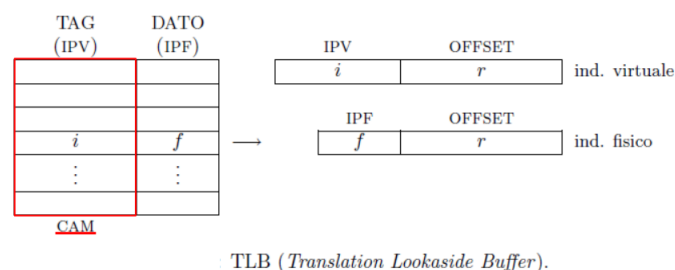
Ad ogni processo è associata una *page table*, l'elemento di indice *i* nella page table corrisponde alla *i*-esima pagina (dello spazio di indirizzi logici, che possiamo ora chiamare indirizzi virtuali) del processo e contiene:

- *IPF*, ovvero l'indice del frame (pagina fisica) nella memoria fisica in cui è collocata la *i*-esima pagina virtuale del processo
- *ism* è il suo indirizzo in memoria secondaria;
- *P* indica se la pagina è presente nella memoria fisica;
- *M* indica se la pagina è stata modificata durante la permanenza in memoria fisica.

Quando viene generato un indirizzo dal processore, la prima cosa che viene controllata nell'indirizzo è il bit *P*. Se *P* = 1 allora la pagina è presente in memoria fisica e di conseguenza viene prelevato l'indice di pagina fisica indicato dai bit **IPF**, questo viene prelevato all'interno della *page table* e viene seguita la traduzione dell'indirizzo come già visto per la paginazione.

Se *P* = 0 allora la pagina si trova nella memoria secondaria, in questo caso viene utilizzato il campo **ims** per identificare la pagina nel disco e successivamente viene caricata in memoria principale. In questo modo risaliamo al **IPF**.

Usando la page table (tenuta in memoria) per trasformare un indirizzo virtuale in un indirizzo fisico, c'è il problema che ogni accesso alla memoria ne richiede almeno due: uno per accedere alla page table; uno per accedere al dato. Per ridurre i tempi di accesso alla page table, è possibile collocare una sua porzione (corrispondente alle pagine fisiche più usate) in una memoria cache chiamata **Translation Lookaside Buffer (TLB)**:

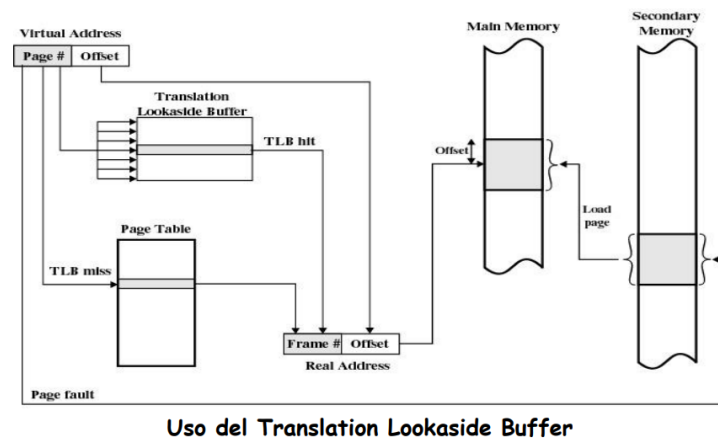


Dato un indirizzo virtuale (page#, offset), il page# (IPV) viene cercato nella memoria associativa (CAM) del TLB:

- se trovato (hit), si estrae il corrispondente frame# (IPF) e si costruisce l'indirizzo fisico (frame#, offset);

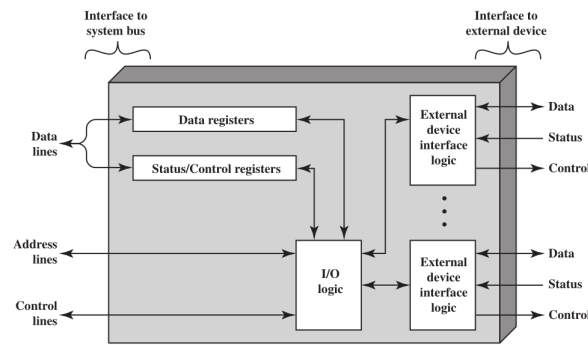
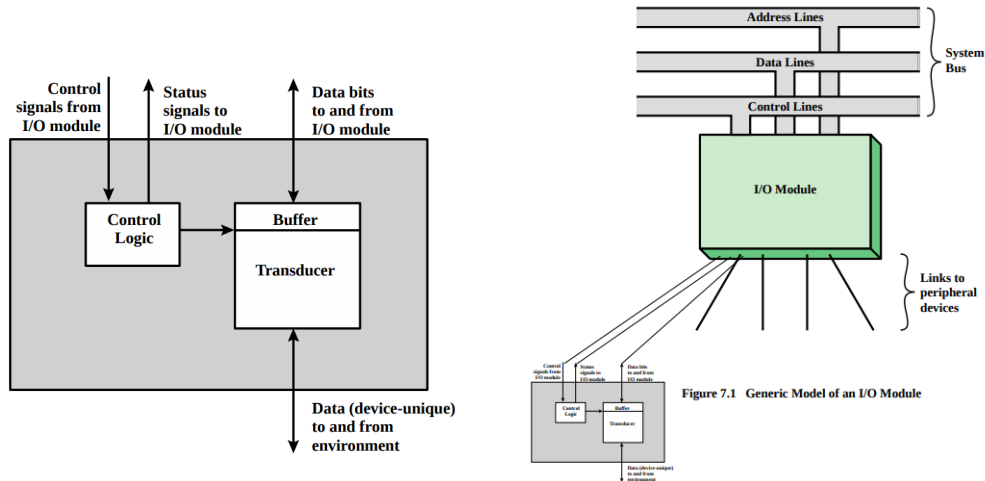
- se non c'è (miss), si accede alla page table (indice=page#):
 - se l'elemento della page table indica che la pagina cercata è presente in un frame nella memoria fisica:
 - * si costruisce l'indirizzo fisico (frame#, offset),
 - * si aggiorna il TLB inserendo l'elemento trovato;
 - altrimenti si ha un page fault (l'indirizzo è in memoria secondaria):
 - * si aggiorna la page table (nuova pagina presente),
 - * si costruisce l'indirizzo fisico (frame#, offset),
 - * si aggiorna il TLB inserendo il nuovo elemento.

Graficamente rappresentiamo il processo nel seguente modo:



5 Input Output

La gestione delle I/O riguarda le operazioni che il processore deve eseguire per comunicare con un dispositivo esterno, al fine di leggere e scrivere dati.

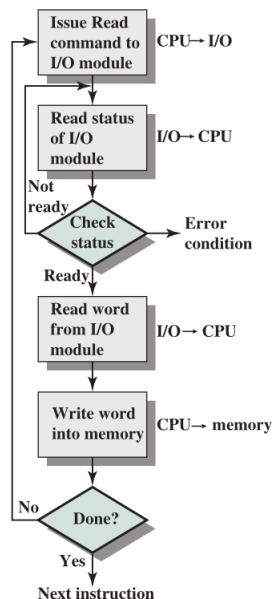


Nella prima immagine viene rappresentato un generico dispositivo esterno. La cpu per comunicare con i dispositivi controlla lo stato, invia segnali di controllo per richiedere la lettura o scrittura, i dati vengono trasferiti tramite un bus dati. Nella seconda immagine viene rappresentato il **modulo di I/O**. Questo modulo è collegato da una parte al nostro elaboratore attraverso il bus di sistema, dall'altro è collegato ad uno o più dispositivi I/O. Attraverso il modulo I/O avviene la comunicazione tra processore e dispositivi esterni, avere un unico modulo che comunica con tutti i dispositivi esterni permette di standardizzare il protocollo di comunicazione. Nella terza immagine vediamo nel dettaglio come è fatto il modulo di I/O. Esistono diversi protocolli che permettono la comunicazione tra elaboratore e dispositivi esterni:

- I/O programmato;
- I/O interrupt;
- I/O DMA.

5.1 I/O programmato

Nel programmed I/O il processore si fa carico direttamente dell'esecuzione delle istruzioni per la lettura dei dati da un dispositivo di input (e la scrittura dei dati su un dispositivo di output).



Nell'immagine viene riportato l'algoritmo per la lettura dei dati da un dispositivo. I dispositivi vengono identificati attraverso un indirizzo, quindi per prima cosa la cpu comunica tale indirizzo al modulo di I/O attraverso le linee di controllo, successivamente viene inviato il comando per richiedere lo status del dispositivo. Il modulo di I/O inoltra il comando al dispositivo il quale comunicherà (se è pronto) la risposta alla CPU attraverso lo status register e le linee dati. Se il dispositivo non è pronto il processore rimane in attesa all'infinito (busy waiting), rimane in ascolto finché non è pronto.

Per accedere al dispositivo:

- **memory mapped:** come se il modulo di I/O fosse mappato in memoria. I dispositivi di I/O sono collegati con le stesse linee del bus usate per la memoria, quindi il processore ci può scrivere con gli stessi comandi con cui accede alla memoria (ldr);
- **isolated:** in questo caso i dispositivi hanno le loro linee dati e di controllo dedicate. I dispositivi hanno uno spazio di indirizzi loro e il processore deve usare istruzioni dedicate per comunicare con i dispositivi.

Il problema di questo protocollo è il busy waiting. Il processore è costantemente impegnato ad aspettare che il dispositivo sia pronto, il ciclo illustrato precedente viene ripetuto milioni di volte. L'I/O programmato può essere utile solo in semplici applicazioni dove il processore non deve gestire più di un dispositivo (per esempio un ascensore).

5.2 I/O a interrupt

Nella gestione ad interrupt è il dispositivo che comunica alla cpu quando è pronto, per farlo invia un segnale, la cpu interrompe il processo che stava eseguendo per gestire questa interruzione dopodiché torna ad eseguire da dove si era interrotta.

Nel dettaglio, per attuare questo protocollo il modulo di I/O deve essere in grado di generare il segnale che avverte il processore che il dato è pronto (IRQ). Il processore deve interrompere quello che stava facendo (quindi salvare il contesto), deve passare velocemente a leggere il dato pronto (RSI) e infine deve ritornare a quello che stava facendo prima dell'interruzione (ripristino contesto).

IRQ sta per interrupt request, è una linea che collega il modulo di I/O con un pin del processore. Attraverso l'IRQ viene comunicato quando lo status del dispositivo è pronto. Il processore può anche decidere di non essere interrotto in quanto magari sta eseguendo alcuni programmi prioritari che non possono essere interrotti. D'altro canto non vogliamo che tutti i dispositivi generino un segnale di

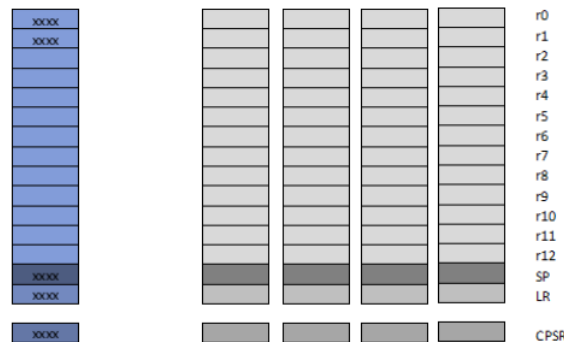
interrupt, per questo viene sfruttato un registro IM che abilita o disabilita il modulo I/O a generare il segnale.

Il salvataggio del contesto è simile ad una chiamata ad una subroutine, dobbiamo salvare PC, registri usati e lo stato del processore (CPSR) in uno stack facendo delle push all'inizio della RSI (se fatto via software).

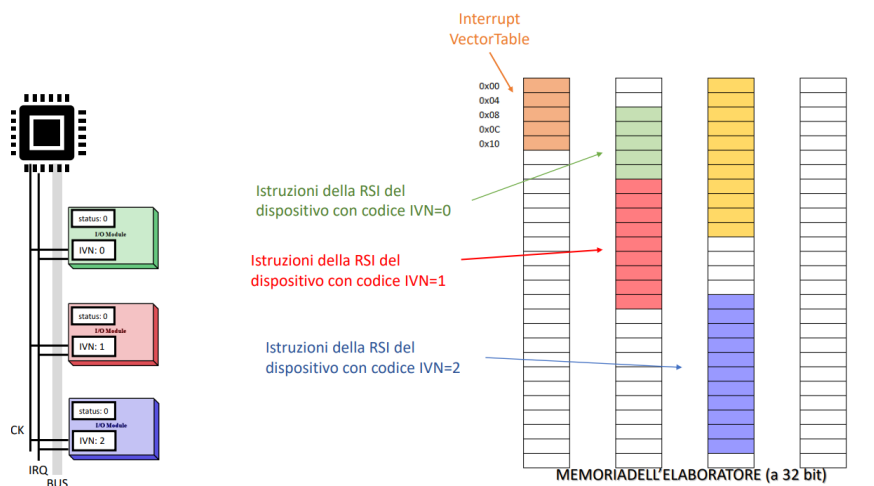
RSI sta per routine di servizio, consiste in un programma software scritto appositamente per la gestione di un tipo di I/O: in un elaboratore con più dispositivi di I/O ci sono più RSI. Se abbiamo più di un dispositivo è quindi necessario capire quale dispositivo di I/O ha generato l'IRQ ed eseguire la RSI corretta.

La domanda che ci poniamo è: come facciamo a identificare il dispositivo che ha generato l'IRQ? Se vengono generati molteplici IRQ, chi ha la priorità? Innanzitutto precisiamo che si possono adottare soluzioni via software o via hardware, ovviamente la seconda opzione è molto più efficiente ma più costosa. In particolare:

- **Salvataggio dati:** abbiamo già anticipato che è possibile sfruttare uno stack per salvare il PC, i registri usati e il CPSR (via software). Una soluzione più efficiente consiste nello sfruttare i **banked registers**. Questi costituiscono un nuovo set di registri dedicati solamente al dispositivo esterno, in questo modo non dobbiamo preoccuparci di salvare i dati su cui stava operando il processore prima di interrompersi. Poiché una RSI può essere interrotta solo da IRQ più prioritari, ci basta aver un set di registri pari al numero di priorità.



- **Identificazione:** Dobbiamo fare un poll software, cioè controllare tutti i registri status dei device di I/O e poi fare un branch alla prima istruzione della RSI (via software). Via hardware sfruttiamo **interrupt vettorizzati**: il processore, in risposta al segnale IRQ, risponde con un segnale IACK (Irq ACKnowledge) che viene inviato a tutti i dispositivi. Solo il dispositivo pronto invierà sulla linea dati un suo codice identificativo che indica la riga da guardare in una tabella che contiene l'indirizzo di tutte le RSI.



- **Gestione della priorità:** un meccanismo di gestione della priorità è in grado, a fronte di più richieste di IRQ, di sceglierne una. Inoltre, se stiamo già gestendo un interrupt, vogliamo che solo gli IRQ con priorità maggiore di quella attualmente servita possano interromperla (quanto visto fin'ora permette di gestire senza problemi anche queste interruzioni di interruzioni). Possiamo scegliere la versione via software e quindi di settare il bit IM (visto in precedenza) che ci permette di scegliere quali dispositivi possono lanciare interrupt. La soluzione via hardware è il **mascheramento**: se abbiamo più linee di IRQ che entrano nel processore, con delle semplici porte logiche possiamo fare in modo che un IRQ a priorità maggiore mascheri i segnali di IRQ di priorità minore. Un'altra possibile soluzione hardware è la **daisy chain**: la priorità è data dall'ordine con cui son collegati i dispositivi. Il processore effettua un hardware pool, cioè genera segnale di IACK e lo invia lungo la linea al primo dispositivo, se questo non ha generato l'IRQ allora passa il segnale al dispositivo successivo, altrimenti ne blocca la propagazione (e invia al processore il suo interrupt vector number IVN).

Il protocollo I/O interrupt è più complesso di quello visto precedentemente, inoltre necessita di software e hardware dedicato. Tuttavia risolve il problema del busy waiting e usando le soluzioni hardware viste abbiamo tempi di risposta rapidissimi.

5.3 I/O con DMA

Parlando di gestione di I/O ci interessa l'impatto sulle performance dell'elaboratore e l'efficienza, ovvero il tempo di risposta. Un altro parametro che ci interessa è il **data rate**, ovvero quanti dati al secondo si possono trasferire. In un sistema a interrupt in cui la gestione dell'I/O in totale richiede $10\mu s$ (tempo di esecuzione della RSI, salvataggio del contesto compreso), quanti dati ci permette di trasferire?

Ad ogni esecuzione la RSI gestisce un tot limitato di dati, se una RSI impiega $10\mu s$ per essere eseguita allora potrà essere eseguita al massimo 100 mila volte in un secondo. Se poniamo che ad ogni esecuzione la RSI gestisce ad esempio 16 bit, il limite massimo del data rate sarà di $200KB/s$. Il problema è che se il dispositivo produce esattamente 100 mila dati al secondo allora processore sarà costantemente impegnato a gestire tale dispositivo, se produce più di 100 mila dati al secondo il processore non riuscirà a leggerli tutti e avremo una perdita di dati.

Il problema è che il processore viene interrotto ogni volta che una piccola parte di dati è pronta, quindi non possiamo sfruttare la gestione ad interrupt vista in precedenza.

La soluzione è il DMA (Direct Memory Access): aggiungiamo al nostro elaboratore un **DMA controller**, questo avrà accesso ai dispositivi esterni e alla memoria tramite bus. Il processore deve solo specificare quanti dati vuole leggere, da che dispositivo e la posizione iniziale dove scriverli in memoria. Il DMA rimarrà in busy waiting sul dispositivo e trasferirà il dato appena pronto; quando il contatore dati si sarà azzerato, comunicherà al processore con un IRQ che il trasferimento è terminato. Possiamo vedere il DMA controller come una sorta di piccola CPU.

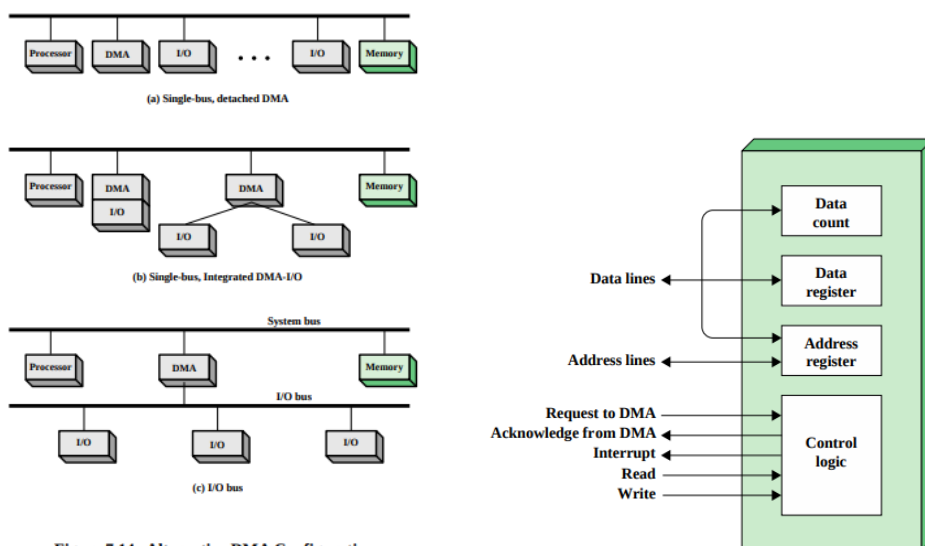


Figure 7.14 Alternative DMA Configurations

Per trasferire i dati in memoria il DMA deve passare per un bus che però è condiviso con il processore, questi potrebbero ostacolarsi a vicenda e crearci dei problemi in quanto non sono ammesse richieste in parallelo. In generale viene data precedenza al DMA perché la perdita di dati è considerata più grave (arbitraggio del bus). Quando ciò accade, il processore rimane fermo in attesa che finisca il DMA e di fatto si ha che il DMA ruba un ciclo alla CPU (cycle stealing).

Con questo protocollo abbiamo visto che è possibile condividere il bus e la memoria tra due dispositivi: DMA e CPU. Con la stessa tecnica possiamo pensare di condividere queste risorse con altri dispositivi, come CPU multicore per il calcolo in parallelo, GPU, ecc.

6 Organizzazione del processore

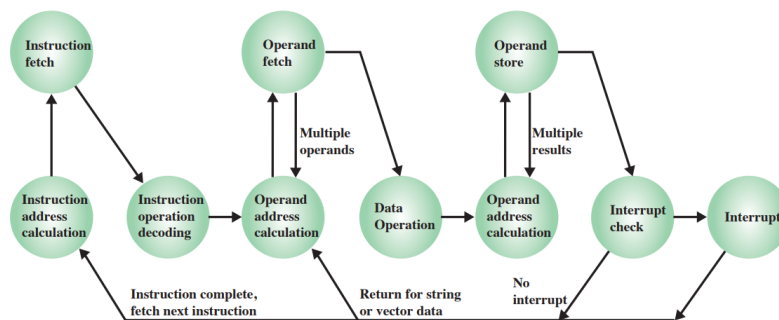
Il ciclo di esecuzione delle istruzioni è composto da tre parti principali: fetch; execute; interrupt. La fetch consiste nei seguenti passaggi:

1. PC contiene l'indirizzo della istruzione successiva;
2. il valore in PC viene spostato in MAR;
3. l'indirizzo viene emesso sul bus degli indirizzi;
4. l'unità di controllo richiede una lettura in memoria principale;
5. il risultato della lettura in memoria principale viene inviato nel bus dati, copiato in MBR, e infine in IR;
6. contemporaneamente alle istruzioni 2-5, il PC viene incrementato.

L'execute dipende dalla istruzione da eseguire, può includere: lettura/scrittura della memoria; input/output; trasferimento di dati fra registri e/o in registri; operazioni della ALU.

Infine gli interrupt: vengono gestiti come abbiamo visto nel capitolo precedente.

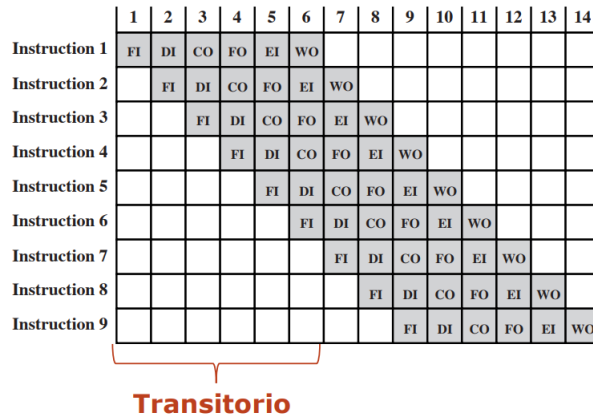
Per velocizzare il processo di esecuzione si può sfruttare l'istruzione **prefetch**: si esegue il fetch di un'istruzione, poi si passa all'execute; prima ancora che sia terminato l'execute si preleva l'istruzione successiva e si esegue il fetch. Questa operazione migliora di poco le prestazioni perché la fase di fetch è molto più veloce dell'execute, in un programma lungo questo approccio non fa la differenza.



Per migliorare consistentemente le prestazioni di un processore viene sfruttata la **pipeline**. Nell'immagine viene rappresentato un ciclo di esecuzione scomposto in 6 fasi l'esecuzione:

1. Fetch instruction (FI): legge la prossima istruzione;
2. Decode instruction (DI): determina l'opcode e il tipo di operandi
3. Calculate operands (CO): calcola l'indirizzo effettivo di ogni operando, gestisce gli indirizzamenti: con offset, diretto, indiretto di registro/memoria,...
4. Fetch operands (FO): recupera gli operandi dalla memoria. Gli operandi nei registri non richiedono il fetch;
5. Execute instruction (EI): esegue l'istruzione indicata
6. Write operand (WO): scrive il risultato in memoria o in un registro;





La pipeline trasforma il processore in una catena di montaggio: viene eseguita la fetch della prima istruzione; viene eseguita la prima istruzione e contemporaneamente viene lanciata la fetch della seconda istruzione; viene eseguita la seconda istruzione e contemporaneamente viene lanciata la fetch della terza istruzione; e così via. Se tutte queste fasi sono eseguite da sezioni indipendenti di hardware, possono essere tutte contemporaneamente attive (su istruzioni diverse). Notiamo che dopo un tempo transitorio iniziale (in cui si esegue la prima istruzione), viene terminata l'esecuzione di un'istruzione al termine di ogni fase.

Tutte le fasi hanno la stessa durata, il periodo di una fase è dato dal periodo massimo delle fasi che sto considerando. Il tempo minimo di avanzamento è dato dalla seguente formula:

$$t = \max_i [t_i] + d$$

dove: k è numero di fasi (6 nell'esempio precedente); t_i è tempo di esecuzione della fase i , con $1 \leq i \leq k$; d è tempo per trasmettere un segnale da una fase alla successiva; t indica il tempo minimo per far avanzare un'istruzione da una fase alla successiva.

Il tempo per eseguire N operazioni è:

$$\text{senza pipeline: } t_1 = N \cdot k \cdot t$$

$$\text{con pipeline: } t_k = k \cdot t + (N - 1) \cdot t$$

$k \cdot t$ è il tempo transitorio (prima istruzione con pipeline vuota), $(N - 1) \cdot t$ indica il tempo di esecuzione di tutte le altre istruzioni.

Per indicare quanto più veloce l'uso di un pipeline rende l'esecuzione di un programma (ovvero di quanto viene ridotto il tempo di esecuzione), rispetto al caso in cui il pipeline sia assente, si calcola il **fattore di speedup** S_k

$$S_k = \frac{T_1}{T_K} = \frac{N \cdot k}{k + N - 1}$$

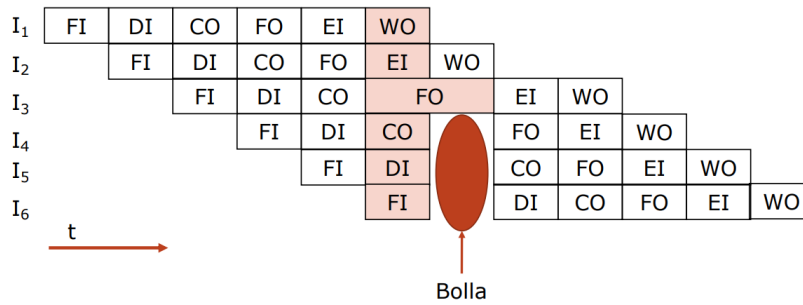
con $N \rightarrow \infty$, lo speedup tende a k cioè al numero di fasi; se $k = 6$ allora il programma girerà 6 volte più veloce rispetto ad una esecuzione senza pipeline.

$$S_k = \lim_{N \rightarrow \infty} \frac{N \cdot k}{k + N - 1} = k$$

Il fattore di speedup S_k è un valore teorico, infatti viene raggiunto solo se il pipeline opera a regime avviando sempre, ad ogni tempo di ciclo, una nuova istruzione (e completandone una). Nella pratica la pipeline può **incepparsi (pipeline stall/bubble)** per diversi motivi:

- **Accessi alla memoria:** le fasi di accesso alla memoria (fetch, lettura e scrittura operandi) hanno la stessa durata delle altre fasi se e solo se si risolvono nella cache, ovvero se provocano un cache hit. In caso di cache miss viene causato un ritardo di 2 o 3 cicli, in questo modo l'esecuzione di tutte le altre istruzioni viene ritardata.

C'è da dire che i cache miss dipendono da come è stato scritto il programma, non sono causati dall'implementazione della pipeline.



- **Conflitti sulle risorse (resource hazard):** ciò accade quando due fasi cercano di accedere alla memoria contemporaneamente. Se le richieste di accesso alla memoria collidono allora una delle due fasi entra in stallo e questo crea un ritardo su tutta la pipeline. Questo tipo di inconveniente è dovuto all'utilizzo della pipeline.
- **Dipendenze tra dati (data hazard):** ciò avviene se per esempio gli operandi di un'istruzione sono i risultati di un'istruzione precedente che deve ancora terminare. Finché l'istruzione precedente non termina, l'istruzione che necessita del dato non può procedere e così si crea un ritardo (*pipeline stall*).

	Clock cycle									
	1	2	3	4	5	6	7	8	9	10
ADD R0, R0, R1	FI	DI	FO	EI	WO					
SUB R1, R1, R0		FI	DI	Idle		FO	EI	WO		
I3			FI			DI	FO	EI	WO	
I4						FI	DI	FO	EI	WO

- **Salti condizionati (branch hazard):** il problema è dovuto al fatto che non si sa in anticipo se la condizione sarà verificata o meno, quindi non sappiamo quale istruzione è quella successiva da eseguire. Se il processore intraprende la strada sbagliata allora bisognerà: svuotare il pipeline; annullare gli effetti delle istruzioni che ne hanno percorso indebitamente alcune fasi; rialimentare il pipeline con le istruzioni dell'altra diramazione.

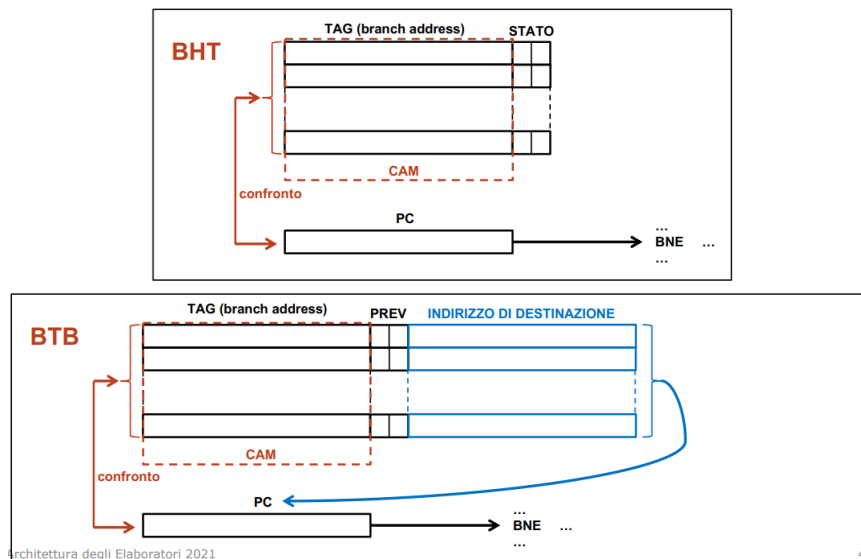
	Time													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO							
Instruction 5					FI	DI	CO							
Instruction 6						FI	DI							
Instruction 7							FI							
Instruction 15								FI	DI	CO	FO	EI	WO	
Instruction 16									FI	DI	CO	FO	EI	WO

Il problema dei *data hazard* può essere risolto riordinando le istruzioni: si sfruttano i criteri di ottimizzazione in fase di compilazione del codice. Le conseguenze negative di un *data hazard* possono essere ridotte dal processore sfruttando la tecnica del **by-pass** (detta anche *data-forwarding*): i risultati prodotti dall'ALU vengono inoltrati allo stadio successivo del pipeline, in contemporanea alla (e senza attendere la) loro memorizzazione.

I *branch hazard* possono essere risolti in diversi modi:

- **multiple streams:** sfruttiamo due pipeline, in questo modo non si può sbagliare. In una viene intrapreso il salto, nell'altra no. Questa soluzione è molto onerosa in termini di costi in quanto richiede sostanzialmente un processore in più per poter gestire la seconda pipeline;
- **delayed branch:** il compilatore riordina le istruzioni in modo tale che prima del salto vengano eseguite solo quelle che non dipendano da esso, cioè le istruzioni che vengono eseguite indipendentemente dal fatto che la condizione si verifichi. Se queste istruzioni non ci sono o sono abbastanza, allora vengono inserite delle NOP (no operation) che comporta un ritardo, ma evita di avviare l'esecuzione di istruzioni che non devono essere eseguite;
- **branch prediction:** si predice quale possa essere il ramo da intraprendere. La scelta può essere:
 - + *statica:* la scelta non dipende dalla storia di esecuzione. Il branch viene sempre eseguito oppure non viene mai eseguito;
 - + *dinamica:* la scelta dipende dalla storia di esecuzione. La scelta di intraprendere o meno il branch dipende da alcuni bit di stato. Lo stato di un branch condizionato viene salvato nella **Branch History Table** (BHT), questa è una piccola cache composta da poche righe che contiene (in una memoria CAM) gli indirizzi delle istruzioni di salto e il loro stato di predizione. Sostanzialmente durante l'esecuzione di un programma: il PC viene confrontato con gli indirizzi nella CAM; se presente, la predizione viene fatta con lo stato di predizione, se assente si inserisce una nuova riga nella tabella e quando il risultato del branch è noto, si aggiorna lo stato di predizione.

Siccome il BHT non ha informazioni sull'indirizzo del branch condizionato (quindi l'istruzione del branch va parzialmente eseguita per calcolare l'indirizzo), si può usare una **Branch Target Buffer** (BTB) che è un BHT in cui ogni riga include anche l'indirizzo del branch.



Perché studiamo ARM? L'architettura ARM è caratterizzata da opcode di lunghezza fissa, anche l'esecuzione delle istruzioni viene resa quanto più possibile fissa. In questo modo abbiamo visto è possibile implementare pipeline molto efficienti.

L'architettura x86 ha opcode di lunghezza variabile e anche il tempo di esecuzione delle istruzioni varia molto, in questo modo è difficile implementare la pipeline in quanto ci sarebbero problemi di pipeline bubble molto frequenti. Per questo motivo i processori che utilizzano l'architettura x86, all'interno, scompongono e riscrivono le istruzioni in un linguaggio molto simile ad ARM al fine di poter implementare e sfruttare la pipeline in modo efficiente.